МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Белгородский государственный технологический университет
им. В.Г. Шухова

ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ

Методические указания к выполнению
лабораторных работ для студентов очной формы
обучения направлений бакалавриата
09.03.02 - Информационные системы и технологии

Белгород
2025

УДК 007(07)
ББК 32.81я7
И74

Составители: %%%

И 74 **ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ**: методические указания к выполнению лабораторных работ для студентов очной формы обучения направлений бакалавриата
09.03.02 - Информационные системы и технологии – Белгород: Изд-во БГТУ им. В.Г. Шухова, 2025. – 114 с.

Методические указания составлены в соответствии с учебным планом и рабочей программой, предназначены для приобретения студентами базовых навыков в работе с криптографическими алгоритмами, содержат теоретический материал и задания к выполнению восьми лабораторных работ.

Методические указания предназначены для студентов очной формы обучения 09.03.02 «Информационные системы и технологии»

УДК 007(07)
ББК 32.81я7

© Белгородский государственный технологический университет (БГТУ) им. В.Г. Шухова, 2025

Оглавление

Лабораторная работа № 1. Классические шифры подстановки 4

Лабораторная работа № 2. Классические шифры перестановки..... 16

Лабораторная работа № 3. Стандарт симметричного шифрования AES 22

Лабораторная работа №4. Генерация простых чисел. Алгоритмы проверки принадлежности простым числам. 55

Лабораторная работа №5.Алгоритм обмена ключами Диффи—Хеллмана. Схема Эль-Гамаля..... 67

Лабораторная работа №6.Комбинированные алгоритмы шифрования 78

Лабораторная работа №7. Алгоритмы контрольных сумм. Электронная подпись 86

Лабораторная работа №8. Криптография эллиптических кривых 98

Правила выполнения лабораторных работ

По курсу «Информационная безопасность» предусмотрено выполнение восьми лабораторных работ. Перед выполнением каждой работы студент обязан самостоятельно ознакомиться с теоретическим материалом и по результатам её выполнения предоставить отчет. Все отчеты о выполнении оформляются на листах формата А4:

Ширина – 21 см. Высота – 29,7 см.
Междустрочный интервал – 1.15.
Интервал после абзаца – 10 пт.
Поля:
Левое – 3 см;
Правое – 1 см;
Верхнее – 2 см;
Нижнее – 2 см.
Шрифт – Times New Roman.
Кегль – 12/14 пт. Цвет – чёрный.
Рисунки, листинги кода и таблицы сопровождаются нумерацией.

Каждый отчет должен содержать:

1. Заголовок:
   1.1. Номер работы и её название;
   1.2. Данные о студенте – ФИО и группа;
   1.3. Две графы – «Выполнение» и «Защита»;
   1.4. Цель работы.
2. Содержание работы и индивидуальные задания.
3. Основные понятия - краткий конспект изученного материала.
4. Ход работы – краткое описание последовательности действий, произведенных при выполнении работы. Эта часть отчёта должна сопровождаться следующими пунктами:
   4.1. Основные составляющие разработанной программы.
   4.2. Блок-схема разработанного алгоритма.
   4.3. Снимки экрана с демонстрацией действия программы.
5. Вывод.
6. Ответы на контрольные вопросы.

Для защиты лабораторной работы необходимо иметь распечатанный отчет и программу на электронном носителе.

Лабораторная работа № 1

КЛАССИЧЕСКИЕ ШИФРЫ ПОДСТАНОВКИ

Цель выполнения работы заключается в формулировании понятия классических криптографических алгоритмов подстановки и в рассмотрении основных принципов их криптоанализа.

Краткие теоретические сведения

Основополагающим понятием в криптографии выступает представление о сокрытом тексте.

Сокрытый текст представимых систем подразумевает наличие исходного текста.

Исходный текст представляет сообщение, которое требуется зашифровать. Может быть представлен в текстовой или иных формах. От данного понятия может быть выведено упомянутое выше – сокрытый текст является результатом применения алгоритма шифрования к исходному тексту.

Алгоритм шифрования выражен принципом, по которому ведётся формирование сокрытого текста из исходного. Алгоритм может быть выражен параметрическим семейством относительно ключа.

Ключом выступает параметр, определяющий конкретный вариант поведения алгоритма шифрования. Ключ может представлять произвольный математический объект, в том числе числовой, составной или любой иной формы.

Текст полагает наличие алфавита. Алфавит выражает множество допустимых значений единицы текста. Алфавит имеет мощность, выражающую число элементов в нём.

Далее для удобства будут приняты следующие принципы обозначений:

P	Исходный текст (plaintext);
C	Сокрытый текст (ciphertext);
k	Ключ шифрования;
$E_k(x)$	Алгоритм шифрования;
$D_k(x)$	Алгоритм дешифрования;
Z_n	Мощность алфавита.

Принципы Керкгоффса

Принципы Керкгоффса постулируют о том, что криптосистема должна представлять параметрическое семейство алгоритмов преобразования данных, конкретизируемое ключом. Безопасность системы должна быть реализуема сокрытием лишь ключа. Если семейство алгоритмов преобразования данных также требуется держать в секрете, то криптосистема представляется достаточно слабой, так как её саботаж потребует однократной компрометации лишь самого алгоритма.

Криптоанализ

Криптоанализ – деятельность, направленная на получение доступа к сокрытой в ходе шифрования информации без априорного представления обо всех составляющих системы сокрытия.

Рассмотрение предмета криптоанализа позволяет обрести навыки проведения оценки того, насколько безопасна система, разрабатываемая рассматривающим лицом.

Криптоанализ вводит понятие криптографической атаки. Данное понятие указывает на сессию осуществления доступа к данным.

Выделяют четыре общих вида атак:

- Атака на поток данных
- Атака с представлением системы
- Атака с исходным текстом
- Атака с исходным текстом и представлением

Первый класс атаки подразумевает лишь наличие данных, порождаемых системой. Представление о системе и исходных данных отсутствуют. Такая атака может зиждиться на выявлении общих паттернов, тем не менее, является малоэффективной.

Второй класс атаки выражает наличие представления об алгоритмах криптосистемы и доступ к потоку данных. Данный вид атаки позволяет провести анализ потока в сопоставлении с алгоритмами, что представляется более эффективным подходом.

Третий класс атаки выражает наличие информации об исходном тексте. Цель данной атаки выражается в получении представления об алгоритме шифрования.

Четвёртый класс выражает наличие исходного текста и представления системы. Цель данного класса атак заключается в обретении сокрытой части системы, позволяющей расшифровывать дальнейший поток сокрытого текста.

Статистическая атака

Данный вид атак рассматривает сокрытый текст, которому пытается сопоставить исходный. Предполагается, что исходный текст является стилистически подобным базовому тексту, для которого посчитаны частоты символов. Аналогичная операция подсчета частот выполняется для рассматриваемого сокрытого текста. Далее ведётся сопоставление частот. Для достаточного объема текстов, отличие в частотах будет составлять порядок одной тысячной.

Основное условие эффективности данного вида атак заключается в том, что на протяжении всего текста одному и тому же исходному символу сопоставлялся один и тот же сокрытый, что аналогично определению моноалфавитного шифра.

Нетрудно продемонстрировать, что при условии рассмотрения полиалфавитного шифра, где имеется доступ к функциональному представлению преобразований, зависящих от позиции, статистическая атака также может быть эффективной – достаточно провести предварительные преобразования сокрытого текста с учетом зависимости.

Метод Казиского

Метод криптоанализа, учитывающий то, что ключ в ряде полиалфавитных шифрах является повторяющимся. На основании этого факта производится группировка текста по столбцам. Число столбцов совпадает с длиной ключа. Каждый столбец будет моноалфавитным, и к нему можно будет применять методы криптоанализа моноалфавитных шифров. Также метод описывает подбор длины ключа на основании минимального расстояния между повторяющимися подстроками d. Исходя из предположения, что повторение подстрок свидетельствует о повторении ключа и исходного текста, строятся варианты длины ключа, являющиеся делителями d.

Категории алгоритмов подстановки

Алгоритмы подстановки репрезентируют те системы шифрования, которые ставят задачей сопоставления блокам символов другие блоки. При этом каждый блок в таких системах самодостаточен и его представление в сокрытом тексте не зависит от других блоков.

Такие алгоритмы делятся на моноалфавитные – те, которые могут сопоставить каждому символу лишь один иной символ при шифровании. Полиалфавитные шифры вводят зависимость единицы сокрытого текста от его исходной позиции, в результате чего одна и та же единица алфавита может быть зашифрована разным образом.

Далее будут рассмотрены следующие алгоритмы шифрования:

1. Аддитивный шифр
2. Мультипликативный шифр
3. Афинный шифр
4. Шифр Плейфеера
5. Автоключевой шифр
6. Шифр Виженера
7. Шифр Хилла
8. Роторный шифр
9. Фонетический алфавит НАТО
10. Одноразовый блокнот
11. Машина Энигма

При рассмотрении подразумевается, что символам алфавита сопоставляются их индексы.

Аддитивный шифр

Данный шифр представляет простейшие преобразования циклического сдвига.

Алгоритм шифрования выражен далее: $E_k(x) = (x + k)\ mod\ Z_n$

Дешифрование выражается аналогично: $D_k(x) = (x - k)\ mod\ Z_n$

Мультипликативный шифр

Мультипликативный шифр производит действия, аналогичные действиям предыдущего алгоритма. Единственное отличие – сложение заменяется умножением. В свою очередь для корректной операции

дешифрования требуется наличие обратного ключу шифрования числа в кольце Z_n. Для этого достаточен факт того, что ключ является взаимно простым с мощностью алфавита.

Шифрование: $E_k(x) = (x * k)\ mod\ Z_n$

Дешифрование: $D_k(x) = (x * k')\ mod\ Z_n$

$k * k' mod\ Z_n = 1$

Афинный шифр

Афинный шифр совмещает два предыдущих принципа. В нём шифрование использует обе операции – умножение и сложение.

Шифрование: $E_k(x) = (x * k_m + k_a)\ mod\ Z_n$

Дешифрование: $D_k(x) = ([x - \ k_a] * k'_m)\ mod\ Z_n$

$k_m * k'_m mod\ Z_n = 1$

Шифр Плейфеера

Данный шифр выделяет единицей текста не отдельные символы, а биграммы – пары символов.

Алгоритм основан на преобразовании биграмм с опорой на алфавит, выложенный в квадратную матрицу.

Ключом выступает слово или фраза, не содержащая повторяющихся символов. Данную фразу размещают таблицей обозначенных соотношений сторон. Если число символов фразы меньше мощности алфавита, она дополняется оставшимися элементами.

Биграммы повторяющихся символов преобразуют, добавляя фиктивный знак. Аналогичное действие совершается при наличии незавершенной биграммы.

Далее, при обозначении индексов строк и столбцов как col_1 и row_1 для первого символа биграммы и col_2, row_2 для второго, можно вывести принципы преобразований:

Случай 1. $col_1 \neq col_2\ row_1 \neq row_2$. Биграммой сокрытого текста в данном случае выступят символы, для которых столбцы и строки измены следующим образом: $col'_1 = col_2\ row'_1 = row_1$ для первого символа и $col'_2 = col_1\ row'_2 = row_2$ для второго. Так, индексы столбцов меняются местами, а индексы строк остаётся неизменными.

Случай 2. $col_1 = col_2$. В данном сценарии индексы строк повышаются на единицу по модулю 5 для английского алфавита, либо размерности

матрицы для других случаев. $row'_i = row_i + 1\ mod\ 5. col'_i = col_i;\ i = 1,2$

Случай 3. $row_1 = row_2$. Аналогичен второму, единственное отличие – теперь повышается индекс столбца.

$col'_i = col_i + 1\ mod\ 5. row'_i = row_i;\ i = 1,2.$

Не трудно представить себе обратные преобразования.

Пример представлен ниже:

Ключ:

kfmpq sunlight ec ab word vxyz

Таблица:

K	F	M	P	Q
S	U	N	L	I
G	H	T	E	C
A	B	W	O	R
D	V	X	Y	Z

Шифрование:

Исходный текст: SOL.

Разбиение на биграммы:

SO L

Добавление фиктивного символа:

SO PL

Первая биграмма:

K	F	M	P	Q
S	U	N	L	I
G	H	T	E	C
A	B	W	O	R
D	V	X	Y	Z

$S \to A$

$O \to L$

Вторая биграмма:

K	F	M	P	Q
S	U	N	L	I
G	H	T	E	C
A	B	W	O	R
D	V	X	Y	Z

$P \to L$

$L \to E$

Шифротекст: ALLE.

Автоключевой шифр

Данный шифр сопоставим с аддитивным шифром. Отличие заключается в том, что шаг сдвига в данном шифре – динамический,

определяется ключом, генерируемым на основании скрываемого сообщения.

Шифрование: $C_i = (S_i + k_{i \bmod m}) \; mod \; Z_n$

Дешифрование: $S_i = (C_i - k_{i \bmod m}) \; mod \; Z_n$

$\exists f\colon \; k \; = \; f(S); N_k = m$

Шифр Виженера

Аналогичен предыдущему, но полагается, что ключ независим от посылаемого сообщения.

Шифрование: $C_i = (S_i + k_{i \bmod m}) \; mod \; Z_n$

Дешифрование: $S_i = (C_i - k_{i \bmod m}) \; mod \; Z_n$

$N_k = m$

Шифр Хилла

Рассматриваемый шифр предполагает разбиение исходного текста на блоки, рассматриваемые далее как вектора. Для получения сокрытого текста вектора умножаются на матрицу-ключ. На матрицу накладываются ограничения наличия обратной к ней в кольце вычетов от Z_n.

$$\begin{pmatrix} C_1 \\ C_2 \\ C_3 \end{pmatrix} = \begin{pmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{pmatrix} \begin{pmatrix} S_1 \\ S_2 \\ S_3 \end{pmatrix}$$

Роторный шифр

Сопоставим с шифром Виженера. Тем не менее, его ключ является динамическим. Изначально представлен перестановкой рассматриваемого алфавита.

$\forall s \in Z\colon \exists! \, s'\colon E(s) = s' \; \wedge \nexists s''\colon E(s'') = \; s'$

Дальнейшее действие алгоритма предполагает обозначающееся вращением действие преобразование функции шифрования:

$\forall s \in Z, s' = E(s)\colon E'(s+1) = s' + 1.$

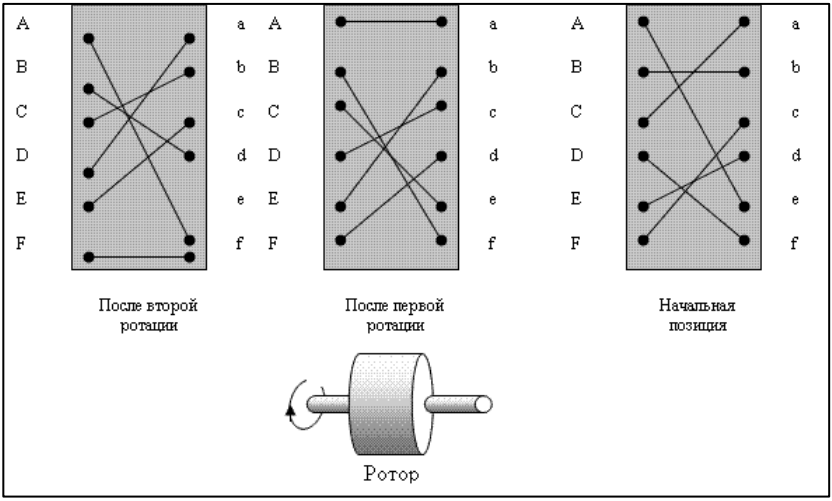


Рисунок 1.1. Иллюстрация к принципу работы роторного шифра.

Фонетический алфавит НАТО

Выражает алгоритм отображения отдельных символов в слова.

Пример:

“..Soul of a land” → “Sierra Oscar Uniform Lima Oscar Foxtrot Alpha Lima Alpha November Delta”

Данный пример наглядно выражает достоверность принципов Керкгоффса – имея представление об алгоритме, получение исходного текста по сокрытому не представляется сложным.

Одноразовый блокнот

Выражает наиболее безопасный алгоритм шифрования – каждому символу сопоставляется единица ключа, позволяющая зашифровать данные тем образом, что расшифровка не будет иметь место, так как сокрытый текст будет изоморфен случайным данным.

Шифрование: $C_i = (S_i + k_i)\ mod\ Z_n$

Дешифрование: $S_i = (C_i - k_i)\ mod\ Z_n$

$N_k = N_S$

Машина Энигма

Включает в рамках обработки данных роторный шифр и табличную подмену, реализуемую через коммутационную панель. Упрощенная схема машины изображена ниже.

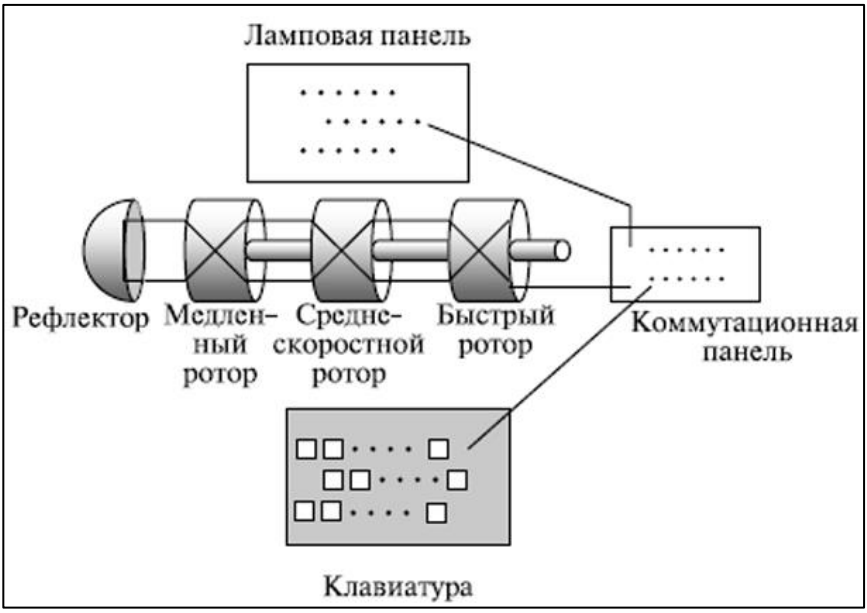


Рисунок 1.2. Схематичное отражение работы машины “Энигма”.

Более детальное рассмотрение механизма может включать выделение следующих составляющих – он включает в себя три ротора, идущий за ним рефлектор и упомянутую коммутационную панель. Три ротора вращаются последовательно – первый – после каждого нажатия на клавиатуру и последующих за этим преобразований, второй – после 26-и поворотов первого, третий – после 26-и поворотов второго. За третьим ротором следует рефлектор. Рефлектор перенаправляет сигнал в обратном порядке сопоставляя каждому символу S на входе символ C на выходе, и, в обратном порядке, символу C на входе, S на выходе. После этого идёт обратный ход сигнала по роторам. В различных исполнениях механизма, рефлектор может также вращаться, либо быть статичным.

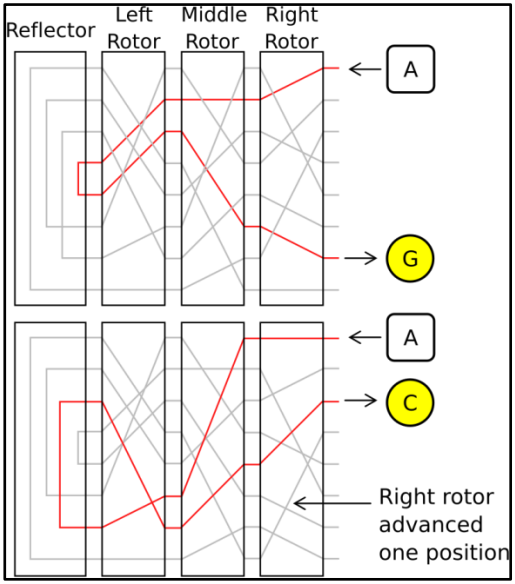


Рисунок 1.3. Принцип действия роторной части.

Коммутационная панель в свою очередь реализует статичную замену символов в начале следования сигнала и при завершении обработки. Она соединет символы в пары. Символ может находиться в паре, либо быть независимым. Если символ находится в паре, при проходе через панель он будет заменён на второй элемент пары.

Содержание работы

1. Реализовать приложение с графическим интерфейсом, включающим возможность применения следующих алгоритмов шифрования:
   1.1. Аффинный шифр.
   1.2. Один из следующих алгоритмов на выбор:
      1.2.1. Шифр Виженера;
      1.2.2. Автоключевой шифр;
      1.2.3. Одноразовый блокнот.
   1.3. *Фонетический алфавит НАТО.
   1.4. *Один из следующих алгоритмов на выбор:
      1.4.1. Роторный шифр;
      1.4.2. Шифр Хилла;
      1.4.3. Шифр Плейфеера;
      1.4.4. **Машина Энигма.

   Пользовательский интерфейс должен поддерживать ввод данных как напрямую через его элементы, так и из файловой системы. Алфавитом выступает следующий набор символов:
   абвгдеёжзийклмнопрстуфкцчшщъыьэюя_,.
2. Провести криптоанализ по следующим пунктам:
   2.1. Для Аффинного шифра – атаку с представлением путём перебора ключей.
   2.2. *Для пункта 1.4. – атаку с исходным текстом и представлением с целью получения ключа/состояния системы.
3. *Провести криптоанализ на основании частоты символов. Для этого:
   3.1. Подобрать исходный текст. Может быть выражен литературным произведением достаточного объема (100 стр.).
   3.2. Преобразовать текст с использованием Аффинного шифра.
   3.3. Сопоставить частоты символов сокрытого текста с частотами, приведёнными в Приложении 1.
   3.4. Произвести дешифровку первых 14 символов сокрытого текста на основании сопоставленных частот.

   Примечание: как было сказано ранее, частоты будут отличаться, на порядок одной тысячной. При этом есть сопоставимые частоты разных исходных символов. В таких случаях следует осуществлять перебор совпадений.

Контрольные вопросы

1. Дать определения следующим понятиям:
   1.1. Моноалфавитный шифр
   1.2. Полиалфавитный шифр
2. Криптоанализ.
   2.1. Определение.
   2.2. Виды атак.
   2.3. Статистическая атака.
3. Принципы работы следующих алгоритмов:
   3.1. Аффинный шифр
   3.2. Шифр Плейфеера
   3.3. Шифр Виженера
   3.4. Автоключевой шифр
   3.5. Одноразовый блокнот
   3.6. Роторный шифр
   3.7. Фонетический алфавит
   3.8. Машина Энигма
4. Охарактеризуйте принцип статистической атаки.
5. Принципы Керкгоффса.
   5.1. Главная позиция
   5.2. Пять дополняющих положений
6. Проведите дешифрование следующего сообщения: " bpnw lphgeew nvprycwkznfqm zdj hsxi n uzqgh dierpr wtthwtof."
   Ключом выступает следующая фраза: "fin d .. scene".
   Символы в позиции точек требуется дополнить самостоятельно, опираясь на сложившуюся картину применения остального ключа.
   Пробелы в тексте и ключе при дешифровании должны быть пропущены.

Лабораторная работа № 2

КЛАССИЧЕСКИЕ ШИФРЫ ПЕРЕСТАНОВКИ

Цель выполнения работы заключается в формулировании представления студента о классических шифрах перестановки и возможностей криптоанализа данных шифров.

Краткие теоретические сведения

Шифры перестановки представляют совокупность алгоритмов, каждый из которых не основывается на постановке отдельным элементам потока исходных данных их сопоставлений в сокрытом тексте, на основании их кода. Вместо этого речь идёт об изменении позиций элементов исходных данных. Речь может идти о простейшей перестановке по ключу, когда поток данных разбивается на блоки, равные длине ключа, и меняет позиции элементов потока в соответствии с этим ключом. Также представимы алгоритмы, не параметризированные ключом и основанные на иных положениях. Ниже будет приведён шифр изгороди, как пример перестановки без ключа.

Криптоанализ шифров обозначенного вида может включать в себя перебор ключей конкретного параметрического семейства шифров. Для продукта человеческой речи в качестве исходного текста могут быть применены и другие подходы, тем не менее, вряд ли уместно говорить о явной формуле, отличной от перебора.

Перестановка с ключом

Данное подмножество имеет своим конкретизирующим параметром ключ, выраженный перестановкой чисел от 1 до n, где n – длина ключа.

Пример функционирования алгоритма представлен ниже:

Исходный текст: "what appears inconceivable.."

Ключ: 3421

w	h	a	t		a	p	p	e	a	r	s		i	n	c	o	n	c	e	i	v	a	b	l	e	.	.
3	4	2	1	3	4	2	1	3	4	2	1	3	4	2	1	3	4	2	1	3	4	2	1	3	4	2	1

t	a	w	h	p	p		a	s	r	e	a	c	n		i	e	c	o	n	b	a	i	v	.	.	l	e
1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4

Сокрытый текст: "tawhpp asreacn ieconbaiv..le"

Дешифрование происходит в обратном порядке действий.

Перестановка обратного ключа

Принцип шифрования, подобный предыдущему. Различие заключается в том, что операции шифрования и дешифрования изменены местами. Нетрудно продемонстрировать, что данный алгоритм является частным случаем предыдущего, изменено лишь представление ключа.

Пример функционирования алгоритма:

Исходный текст: "what appears inconceivable.."

Ключ: 3421.

w	h	a	t		a	p	p	e	a	r	s		i	n	c	o	n	c	e	i	v	a	b	l	e	.	.
1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4

a	t	h	w	p	p	a		r	s	a	e	n	c	i		c	e	n	o	a	b	v	i	.	.	e	l
3	4	2	1	3	4	2	1	3	4	2	1	3	4	2	1	3	4	2	1	3	4	2	1	3	4	2	1

Сокрытый текст: "athwppa rsaenci cenoabvi..el"

В подтверждение предыдущих слов, можно заметить, что действия были бы аналогичны применению предыдущего алгоритма с ключом 3412.

Перестановка без ключа

Данная подгруппа алгоритмов перестановки включает запись текста в таблицу построчно, после чего следует чтение по столбцам. Аналогично можно записывать текст по столбцам и читать по строкам – результат будет идентичным.

Самым простым примером данного алгоритма является шифр изгороди, при котором текст записывается в две строки по столбцам. Чтение идёт поочерёдно по строкам.

Комбинированная перестановка

Данный вид перестановки включает в себя оба предыдущих алгоритма. Если имеется ключ K длиной N, то алгоритм выразим следующим образом:

Строится таблица R x N, где R обуславливается длиной сообщения. Текст в таблицу записывается построчно. Далее столбцы меняются местами в соответствии с ключом, по алгоритму, аналогичному приведённому для перестановки первого вида.

Чтение, затем, проводится по столбцам.

Пример шифрования:

Исходный текст: "what appears inconceivable."

Ключ: 3421.

w	h	a	t
	a	p	p
e	a	r	s
	i	n	c
o	n	c	e
i	v	a	b
l	e	.	
3	4	2	1

t	a	w	h
p	p		a
s	r	e	a
c	n		i
e	c	o	n
b	a	i	v
	.	e	l
1	2	3	4

Сокрытый текст: "tawhpp asreacn ieconbaiv .el"

Дешифрование проходит в обратном порядке – запись текста в столбцы, обратная перестановка столбцов и, затем, чтение текста по строкам.

Кратная перестановка

Применение перестановки с ключом кратное число раз может выступить в качестве основания алгоритма. Тем не менее, нетрудно продемонстрировать, что это будет частным случаем исходного алгоритма с неявной функцией преобразования ключа – так как всем позициям на входе всегда сопоставлена лишь одна позиция на выходе.

Криптоанализ шифров перестановки

Одна из проблем, возникающая перед криптоаналитиком может быть выражена как оптимизация процесса получения ключа. Простейшая атака грубой силы в данном вопросе также может быть саботирована критерием соответствия расшифрованного текста исходному. Одним из вариантов выступает критерий частот биграмм – шифр перестановки не изменяет частоту отдельных символов, но частота биграмм подвергается модификации. Самым простым способом эксплуатации данного факта выступает дешифрование на основе исключения “запрещённых” биграмм – тех, которые не могут быть встречены в тексте на конкретном естественном языке.

Вторым вариантом может выступить минимизация расстояния столбцов диаграммы частот биграмм в дешифрованном тексте, при результатах применения ключа перебора, и средних значений.

Содержание работы

1. Реализовать приложение с графическим интерфейсом, включающее возможность применения следующих алгоритмов шифрования:
   1.1. Шифр изгороди
   1.2. Перестановка с ключом
   1.3. Комбинированная перестановка
   1.4. *Кратная перестановка

   Пользовательский интерфейс должен поддерживать ввод данных как напрямую через его элементы, так и из файловой системы. Алфавитом выступает следующий набор символов:
   абвгдеёжзийклмнопрстуфкцчшщъыьэюяАБВГДЕЁЖЗИЙКЛМНОПРСТУФКЦЧШЩЪЫЬЭЮЯ_,.?!
2. *Разработать алгоритм, который будет перебирать ключ для обозначенных алгоритмов по одному из следующих требований:
   2.1. Достоверность очередного ключа подтверждается или опровергается пользователем при наблюдении фрагмента результата. Вид алгоритма указывается пользователем.
   2.2. Вид алгоритма также указывается пользователем, но достоверность подобранного ключа проверяет программа. Максимальная длина ключа указывается пользователем.
   2.3. Программа автономно перебирает ключи и конкретный вид алгоритма перестановки. Максимальная длина ключа также указывается пользователем.

Контрольные вопросы

1. Дать определение шифра перестановки.
2. Отличительные черты алгоритмов перестановки.
3. Криптоанализ алгоритмов перестановки.
4. Изложить принципы работы следующих алгоритмов:
   4.1. Шифр изгороди.
   4.2. Перестановка с ключом
   4.3. Комбинированная перестановка
   4.4. Кратная перестановка
5. Ключ шифрования для алгоритма перестановки выражен следующим вектором: (7, 1, 2, 4, 5, 3, 6).
   Требуется определить ключ дешифрования.
6. Подберите ключ шифрования для следующего сокрытого текста:
   "atWh_aeapprsnc_ionivceab_mleayav_he_qua_itsie_mp_sleolioutn."
   Семейством алгоритма выступает перестановка с ключом

Лабораторная работа № 3

СТАНДАРТ СИММЕТРИЧНОГО ШИФРОВАНИЯ AES

Цель выполнения работы заключается в формировании представления студента о симметричном шифровании и стандарте шифрования AES-128. Требуется сформировать программу, реализующую обозначенный стандарт.

Краткие теоретические сведения

AES-128 – алгоритм симметричного шифрования, заменивший использовавшийся ранее стандарт DES. Длина блока рассматриваемого алгоритма составляет 128 бит. Размер ключа может быть различным – 128, 192 либо 256 бит. Данный алгоритм является подмножеством семейства алгоритмов Rijndael, конкретизированный относительно размера блока.

Используемая терминология

Байт – последовательность из восьми бит.

Слово – последовательность из четырёх байт.

Блок – последовательность из шестнадцати байтов. Ровно как итоговый сокрытый текст, так и исходный разбиваются на блоки и поставляются алгоритму в разбитом виде. Алгоритм обрабатывает каждый входной блок отдельно, выдавая на выходе объект той же структуры, но в зашифрованном виде.

Основной ключ – последовательность из N_k байт. Число байт N_k определяется конкретным случаем применения алгоритма. Стандарт определяет возможные размеры ключа как 16, 24 и 32 байт (128, 192 и 256 бит)

Матрица состояний – двумерный массив байт, состоящий из четырёх столбцов и четырёх строк. При этом принимается, что байты следуют в матрице по столбцам:

[0]	[4]	[8]	[12]
[1]	[5]	[9]	[13]
[2]	[6]	[10]	[14]
[3]	[7]	[11]	[15]

Следует выделить обстоятельство того, что в следующих далее иллюстрациях к алгоритму полагается, что первый индекс матрицы состояния – индекс столбца, второй – строки.

Раунд – итерация цикла преобразования состояния. Число раундов обозначается как N_r и определяется длиной ключа. Имеют место следующие сопоставления:

N_k	4	6	8
N_r	10	12	14

Ключ раунда – значение, вычисляемое в процедуре KeyExpansion. Представляет отдельное значение для каждой итерации алгоритма. Применяется к матрице состояний путём операции XOR.

Подстановка байт – операция табличной замены входящих потоков данных на их соответствия.

Подстановка слова – аналогичная предыдущей операция, применяемая к словам, рассматривая их как набор из атомарных байт.

Используемые обозначения

N_r – число раундов;

N_k – число слов в ключе;

N_b – число слов во входных данных, для стандартного алгоритма всегда равно четырем.

SBox – таблица подмены байт, использующаяся в операции подстановки.

RCon – таблица, использующаяся при построении расширенного ключа.

Алгоритм AES-128

Алгоритм можно разбить на четыре операции, выполняемые для каждого блока и одну предварительную операцию, общую для всех блоков.

Четыре основные операции:

1. AddRoundKey – операция XOR, применяющаяся между ключом раунда и матрицей состояния.
2. SubBytes – операция подмены байт.
3. ShiftRows – циклический сдвиг строк матрицы.
4. MixColumns – смешивание значений внутри столбцов матрицы состояния.

Предварительная операция: ExpandKey – формирование ключей раунда

Аналогично, обратный алгоритм включает в себя четыре блочные операции. Три из них отличаются от предыдущих, в то время как первая операция является зеркальной и не требует переопределения. Три упомянутые обратные операции будут обозначены как InvSubBytes, InvShiftRows и InvMixColumns.

Псевдокод алгоритма:

```
Encrypt(byte[Nb][4] in, byte[Nb][4] out, word[Nr+1][Nb] rKeys)
begin
  byte state[Nb][4]
  state = in

  AddRoundKey(state, rKeys[0])
  for round = 1 to Nr-1 with step 1
    SubBytes(state)
    ShiftRows(state)
    MixColumns(state)
    AddRoundKey(state, rKeys[round])
  end for
  SubBytes(state)
  ShiftRows(state)
  AddRoundKey(state, rKeys[Nr])

  out = state
end
```

Листинг 3.1. Псевдокод общего алгоритма

Блок схема алгоритма:

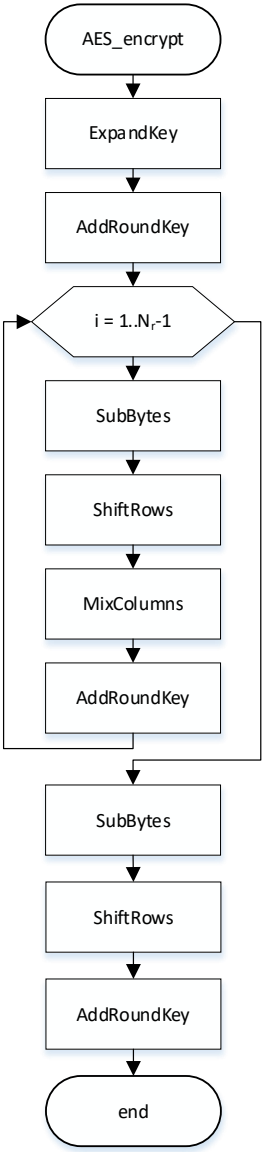


Рисунок 3.1. Блок-схема алгоритма прямого шифрования.

Далее приведено развернутое описание каждого из шагов алгоритма.

KeyExpansion

Операция несёт своей целью построение расширенного ключа по основному.

Псевдокод операции:

```
KeyExpansion(byte[4*Nk] key, word[Nr+1][Nb] rKeys, Nk)
begin
   word temp
   for i = 0 to Nk-1 with step 1
      rKeys[i div 4][i mod 4] = word(key[4*i], key[4*i+1], key[4*i+2], key[4*i+3])
   end for
   for i = Nk to (Nr+1)*Nb-1 with step 1
      r = i div 4; c = i mod 4
      rlast = (i-1) div 4
      clast = (i-1) mod 4
      temp = rKeys[rlast][clast]
      if i mod Nk = 0
         rotated = RotWord(temp)
         substituted = SubWord(rotated)
         temp = substituted xor Rcon[i div Nk]
      else if (Nk > 6 and i mod Nk = 4)
         temp = SubWord(temp)
      end if
      rprev = (i-Nk) div 4
      cprev = (i-Nk) mod 4
      rKeys[r][c] = roundKeys[rprev][cprev] xor temp
   end for
end
```

Листинг 3.2. Псевдокод процедуры KeyExpansion

Примечание: процедура RotWord циклически сдвигает передаваемое слово на 8 бит влево. Процедура SubWord в свою очередь применяет операцию подстановки слова, где для каждого байта в составе слова применяется операция подмены байт. Rcon – табличное значение, приведённое в Приложении 2. Выражает степени двойки в поле Галуа $GF(2^8)$ с порождающим полиномом $c(x) = x^8 + x^4 + x^3 + x + 1$.

Блок схема операции:

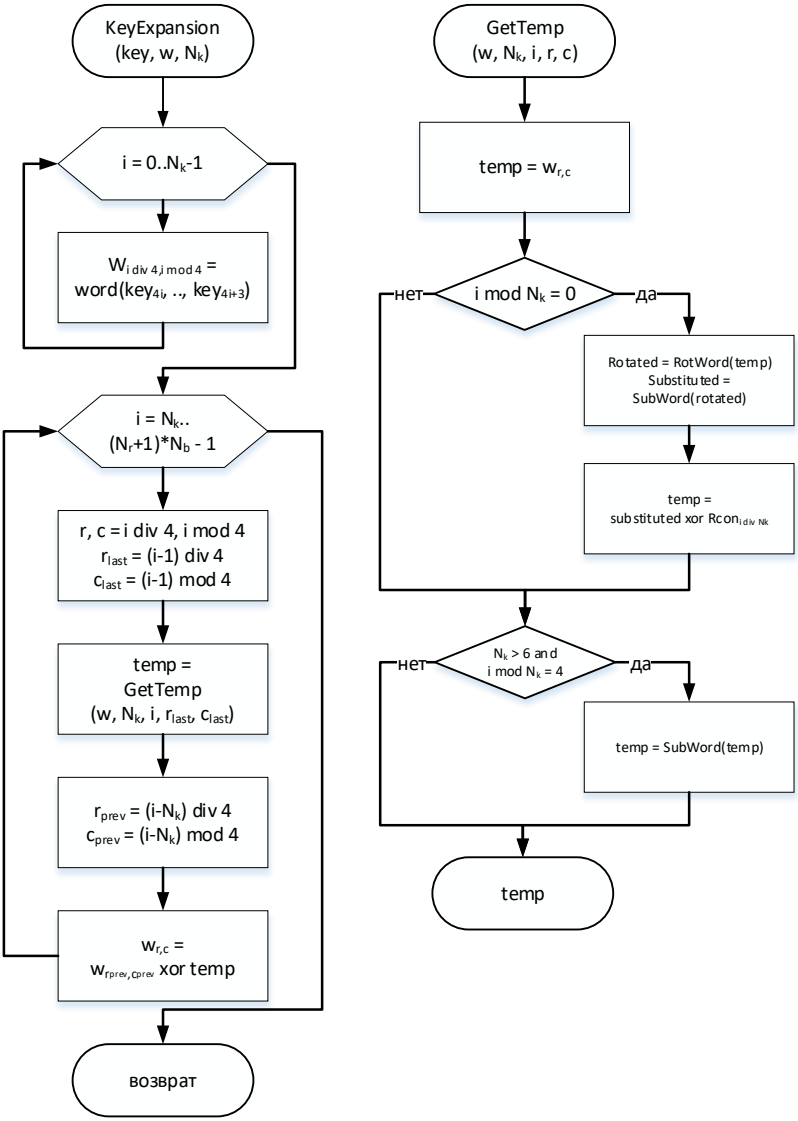


Рисунок 3.2. Блок-схема операции расширения ключа.

AddRoundKey

Данная операция осуществляет применение операции XOR в сопоставлении расширенного ключа с текущей матрицей состояния. Каждое слово в рамках ключа раунда накладывается последовательно на каждый из 4-х столбцов состояния. Первый байт ключа – на первый элемент первой строки, второй байт ключа – на первый элемент второй строки, пятый байт – на второй элемент первой строки, аналогично далее.

Псевдокод операции:

```
AddRoundKey(byte[Nb][4] state, word[Nr+1][Nb] key, int round)
begin
   for i = 0 to Nb-1 with step 1
      tmpWord = word(state[i][0], state[i][1], state[i][1], state[i][3])
      tmpWord = tmpWord xor key[round][i]
      state[i][0] = (tmpWord >> 24) & 0xff
      state[i][1] = (tmpWord >> 16) & 0xff
      state[i][2] = (tmpWord >>  8) & 0xff
      state[i][3] = (tmpWord      ) & 0xff
   end for
end
```

Листинг 3.3. Псевдокод операции применения ключа раунда.

Примечание: оператор x >> n подразумевает сдвиг значения переменной x на n бит вправо. Оператор a & b выражает побитовую операцию конъюнкции.

Блок схема операции:

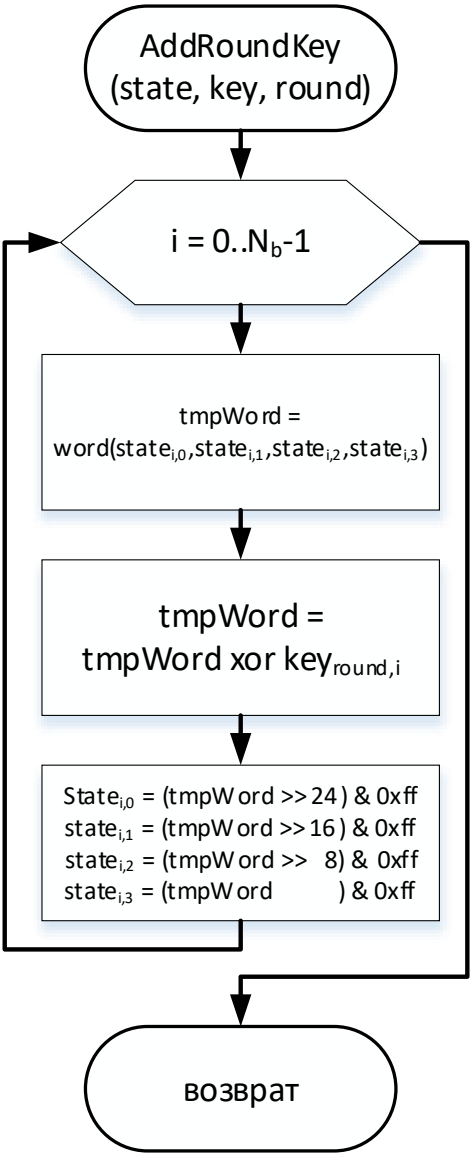


Рисунок 3.3. Блок-схема операции применения ключа.

SubBytes

Обозначенная процедура производит замену байт. Первым действием выполняется взятие мультипликативно-обратного в поле Галуа GF(2^8) по модулю $m(x) = x^8 + x^4 + x^3 + x + 1$. Затем производится операция умножения предопределённой матрицы на вектор бит рассматриваемого значения. Матрица представляет строки циклического сдвига изначально заданной константы на 0, 1, 2, .., 7 бит вправо. После матричного умножения производится применение операции xor второй константы к результату. Данное представление переводимо в упрощенную формулу, представленную ниже:

$$b'_i = b_i \oplus b_{(i+4)\ mod\ 8} \oplus b_{(i+5)\ mod\ 8} \oplus b_{(i+6)\ mod\ 8} \oplus b_{(i+7)\ mod\ 8} \oplus c_i$$

Здесь b' – итоговое значение, b_i – i-й бит значения b, c – константа, равная 63_{16}, 01100011_2, либо 99_{10}.

Матричное представление операции отражено ниже:

$$\begin{pmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{pmatrix} = \begin{pmatrix} 1&0&0&0&1&1&1&1 \\ 1&1&0&0&0&1&1&1 \\ 1&1&1&0&0&0&1&1 \\ 1&1&1&1&0&0&0&1 \\ 1&1&1&1&1&0&0&0 \\ 0&1&1&1&1&1&0&0 \\ 0&0&1&1&1&1&1&0 \\ 0&0&0&1&1&1&1&1 \end{pmatrix} * \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{pmatrix} \oplus \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

Учитывая область определения операции, для оптимизации процесса шифрования уместно использовать предварительно строимую таблицу подмены байт, обозначаемую как SBox. Её значение приведено в приложении 2.

Псевдокод операции:

```
SubBytes(byte[Nb][4] state)
begin
   for i = 0 to Nb – 1 with step 1
      for j = 0 to 3 with step 1
         state[i][j] = SBox[state[i][j]]
      end for
   end for
end
```

Листинг 3.4. Псевдокод операции подстановки байт.

Блок-схема операции:

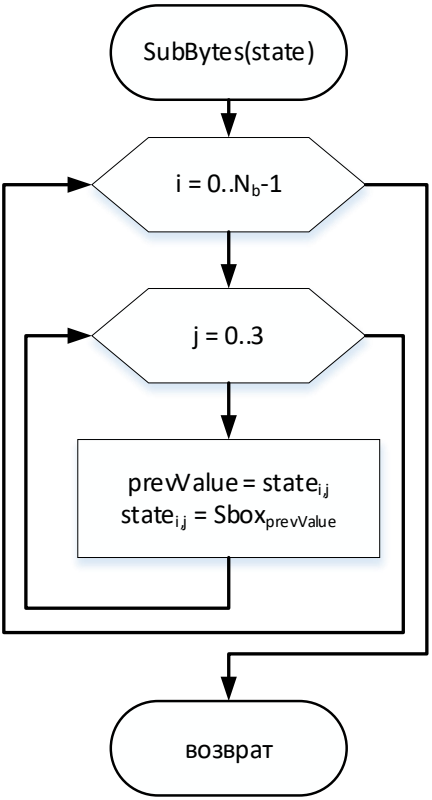


Рисунок 3.4. Блок-схема операции подстановки байт.

ShiftRows

Данная операция подразумевает циклический сдвиг строк таблицы состояния в зависимости от индекса строки. Первая строка не изменяется. Вторая строка сдвигается на один столбец, третья на два, четвёртая, идентично, на три.

Псевдокод операции:

```
ShiftRows(byte[Nb][4] state)
begin
   prevValues = state
   for i = 0 to Nb – 1 with step 1
      for j = 0 to 3with step 1
         colIndex = (i+j) mod Nb
         state[i][j] = prevValues[colIndex][j]
      end for
   end for
end
```

Листинг 3.5. Псевдокод операции сдвига строк.

Блок-схема операции:

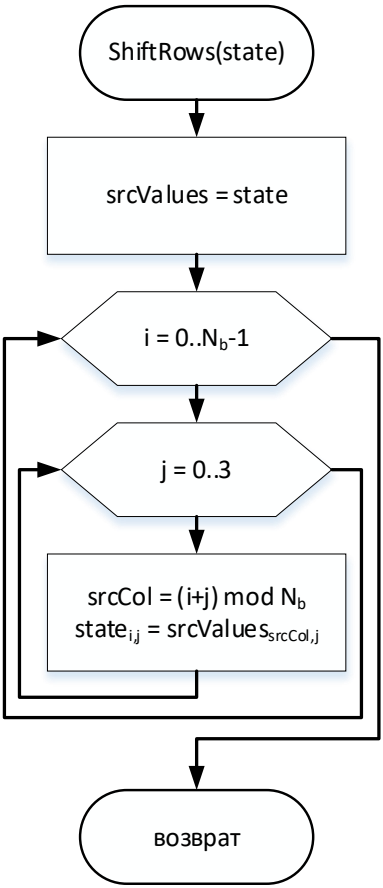


Рисунок 3.5. Блок-схема операции сдвига строк.

MixColumns

Обозначенная операция осуществляет умножение в поле Галуа GF(2^8) с порождающим полиномом g(x) = $x^8 + x^4 + x^3 + x + 1$. Каждый из столбцов принимается за полином третьей степени

$$b(x) = b_3x^3 + b_2x^2 + b_1x + b_0,$$

после чего производится умножение в обозначенном поле на полином $a(x) = 3x^3 + x^2 + x + 2$ по модулю x^4+1.

Для удобства представим a(x) в параметрическом виде:

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0.$$

Далее представим умножение этих полиномов: $c(x) = a(x) * b(x) =$

$$= (a_3x^3 + a_2x^2 + a_1x + a_0) * (b_3x^3 + b_2x^2 + b_1x + b_0) =$$

$$= c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x^1 + c_0 = c(x)$$

где

$c_0 = a_0 * b_0$;

$c_1 = a_1 * b_0 \oplus a_0 * b_1$;

$c_2 = a_2 * b_0 \oplus a_1 * b_1 \oplus a_0 * b_2$;

$c_3 = a_3 * b_0 \oplus a_2 * b_1 \oplus a_1 * b_2 \oplus a_0 * b_3$;

$c_4 = a_3 * b_1 \oplus a_2 * b_2 \oplus a_1 * b_3$;

$c_5 = a_3 * b_2 \oplus a_2 * b_3$;

$c_6 = a_3 * b_3$.

Так как произведение выполняется по модулю, полином $c(x)$ преобразуется по следующему принципу:

$x^6 \ mod \ (x^4 + 1) = -x^2 = x^2$;

$x^5 \ mod \ (x^4 + 1) = -x = x$;

$x^4 \ mod \ (x^4 + 1) = -1 = 1$.

В таком случае выполняется следующее соответствие:

$c(x) \ mod \ (x^4 + 1) = d_3x^3 + d_2x^2 + d_1x + d_0$, где

Слагаемое	Выражение	Выражение от исходных данных
d_3	c_3	$a_3 * b_0 \oplus a_2 * b_1 \oplus a_1 * b_2 \oplus a_0 * b_3$
d_2	$c_2 \oplus c_6$	$a_2 * b_0 \oplus a_1 * b_1 \oplus a_0 * b_2 \oplus a_3 * b_3$
d_1	$c_1 \oplus c_5$	$a_1 * b_0 \oplus a_0 * b_1 \oplus a_3 * b_2 \oplus a_2 * b_3$
d_0	$c_0 \oplus c_4$	$a_0 * b_0 \oplus a_3 * b_1 \oplus a_2 * b_2 \oplus a_1 * b_3$

Взглянув на таблицу, можно заметить, что каждый множитель итогового результата содержит значения столбца, умноженные на один из коэффициентов полинома a.

Это преобразование удобно представить в виде матричного умножения матрицы, порождаемой a(x), на каждый из столбцов:

$$s'_i = \begin{pmatrix} s'_{i,1} \\ s'_{i,2} \\ s'_{i,3} \\ s'_{i,4} \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \times \begin{pmatrix} s_{i,1} \\ s_{i,2} \\ s_{i,3} \\ s_{i,4} \end{pmatrix},$$

где s'_i – столбец с индексом i.

Псевдокод операции:

```
MixColumns(byte[Nb][4] state)
begin
   matrix = array{
      array{02,03,01,01},
      array{01,02,03,01},
      array{01,01,02,03},
      array{03,01,01,02}
   }
   for i = 0 to Nb with step 1
      column = state[i]
      newValue = 0
      for j = 0 to 3 with step 1
         cellRes = matrix[i][j]*column[j]
         newValue = newValue xor cellRes
      end for
      state[i] = GaloisMod(newValue, 0x11b)
   end for
end
```

Листинг 3.6. Псевдокод операции смешения столбцов.

Блок-схема процедуры:

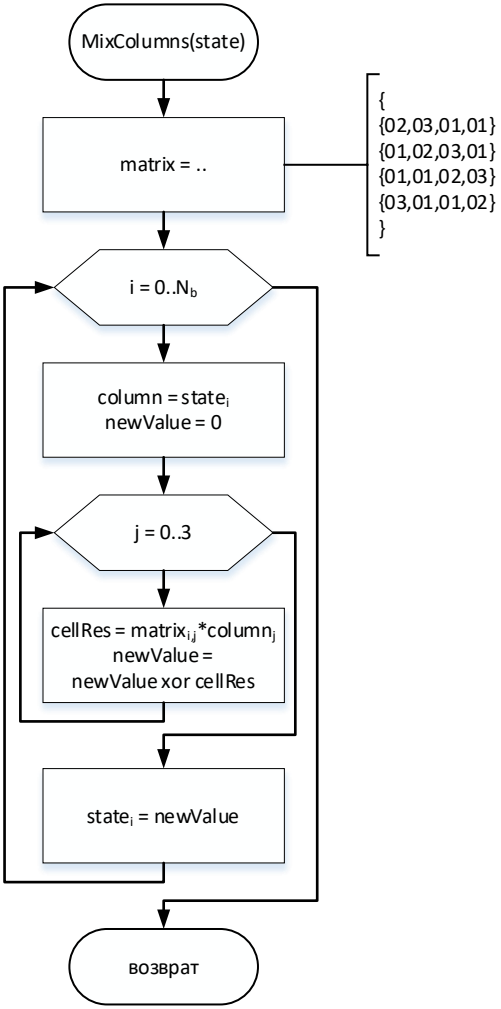


Рисунок 3.6. Блок-схема операции смешения столбцов.

Дешифрование

Дешифрование включает в себя обратные версии операций. Операция применения ключа идентична обратной, так как операция XOR является симметричной при идентичном втором операнде.

Блок-схема алгоритма дешифрования:

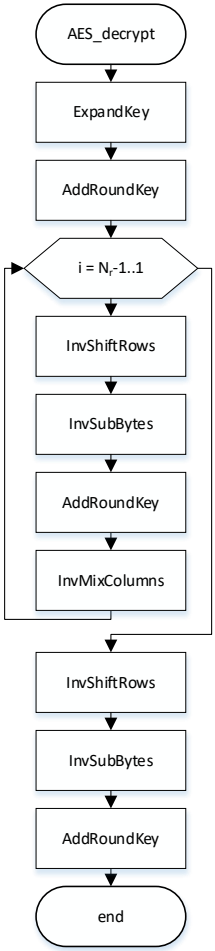


Рисунок 3.7. Блок-схема алгоритма дешифрования.

Псевдокод алгоритма дешифрования:

```
Decrypt(byte[Nb][i] in, byte[Nb][i] out, word[Nr+1][Nb] rKeys)
begin
   byte state[Nb][4]
   state = in

   AddRoundKey(state, rKeys[Nr])

   for round = Nr - 1 downto 1 with step -1
      InvShiftRows(state)
      InvSubBytes(state)
      AddRoundKey(state, rKeys[round])
      InvMixColumns(state)
   end for

   InvShiftRows(state)
   InvSubBytes(state)
   AddRoundKey(state, rKeys[0])

   out = state
end
```

Листинг 3.7. Псевдокод алгоритма дешифрования.

InvShiftRows

Данный вариант операции не имеет принципиальных отличий от его прямого варианта. Исключением выступает лишь направление – в данном исполнении элементы перемещаются вправо.

Ниже приведен псевдокод операции:

```
InvShiftRows(byte[Nb][4] state)
begin
   prevValues = state
   for i = 0 to Nb-1 with step 1
      for j = 0 to 3 with step 1
         colIndex = (Nb+i-j) mod Nb
         state[i][j] = prevValues[colIndex][j]
      end for
   end for
end
```

Листинг 3.8. Псевдокод операции обратного сдвига строк.

Далее приведена блок-схема рассмотренной операции:

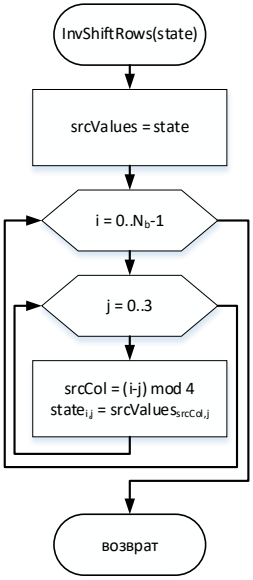


Рисунок 3.8. Блок-схема операции обратного сдвига строк

InvSubBytes

Операция обратной подстановки байт отличается от основной версии принципом построения таблицы подстановок – она достигается следующим соотношением:

$$b'_i = b_{(i+2)\ mod\ 8} \oplus b_{(i+5)\ mod\ 8} \oplus b_{(i+7)\ mod\ 8} \oplus \mathrm{c}'_i\ ,$$

где $\mathrm{c}' = 05_{16}$.

После чего, также как в прямом направлении шага, идёт взятие мультипликативно-обратного в поле Галуа GF(2^8) по модулю $m(x) = x^8 + x^4 + x^3 + x + 1$. Отличием здесь представлен порядок действий – в прямом направлении инверсия производится перед основным преобразованием.

Табличные значения массива InvSbox, получаемого в результате применения обозначенных преобразований приведены в Приложении 2.

Далее приведена блок-схема рассмотренной операции:

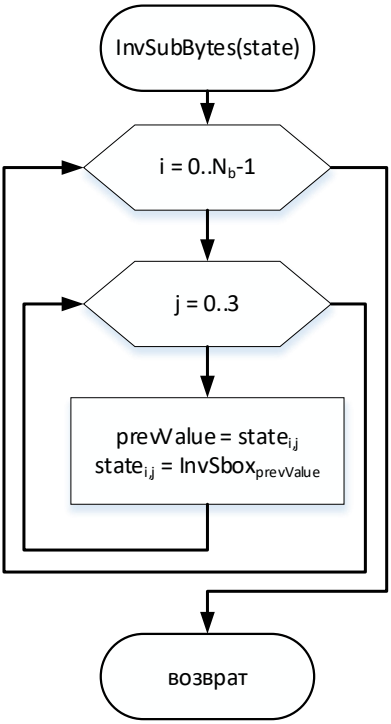


Рисунок 3.9. Блок-схема операции обратной подмены байт.

Псевдокод рассмотренной операции приведен далее:

```
InvSubBytes(byte[Nb][4] state)
begin
   for i = 0 to Nb-1 with step 1
      for j = 0 to Nb with step 1
         state[i][j] = InvSBox[state[i][j]]
      end for
   end for
end
```

Листинг 3.9. Псевдокод операции обратной подмены байт

InvMixColumns

Операция обращения смешения строк осуществляется также, как исходная операция, за исключением того, что умножение происходит на иную матрицу. Выражение принимает следующий вид:

$$s' = \begin{pmatrix} s'_1 \\ s'_2 \\ s'_3 \\ s'_4 \end{pmatrix} = \begin{pmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{pmatrix} \times \begin{pmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{pmatrix}$$

Псевдокод операции приведен далее:

```
InvMixColumns(byte[Nb][4] state)
begin
   matrix = array{
      array{0e,0b,0d,09},
      array{09,0e,0b,0d},
      array{0d,09,0e,0b},
      array{0b,0d,09,0e}
   }
   for i = 0 to Nb - 1 with step 1
      column = state[i]
      newValue = 0
      for j = 0 to 3 with step 1
         cellRes = matrix[i][j]*column[j]
         newValue = newValue ^ cellRes
      end for
      state[i] = newValue
   end for
end
```

Листинг 3.10. Псевдокод операции обратного смешения строк.

Блок-схема операции приведена ниже:

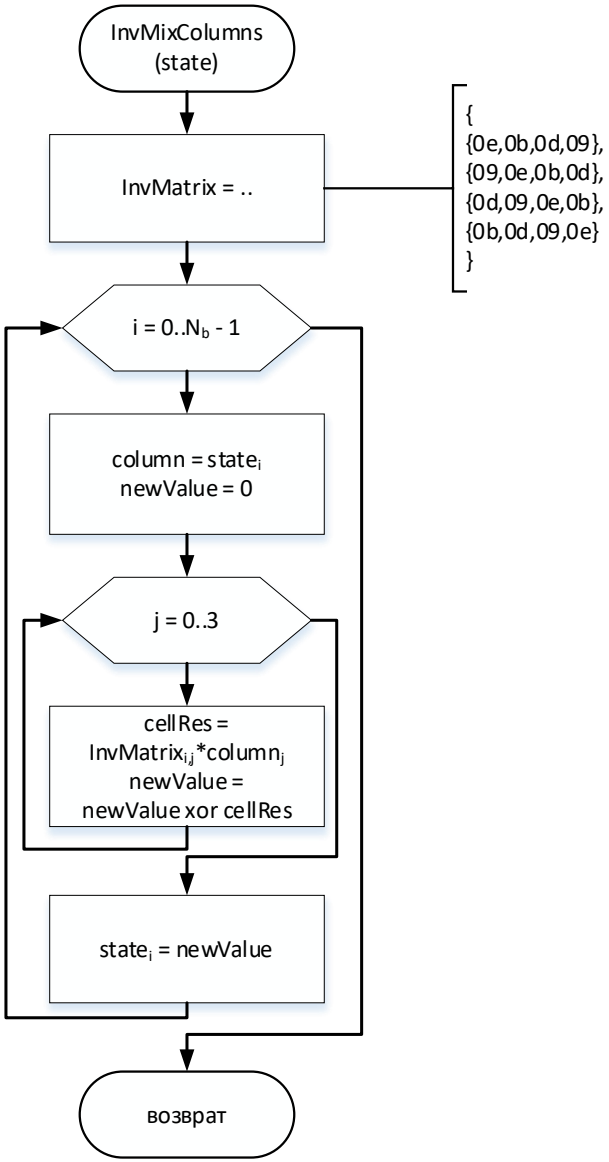


Рисунок 3.10. Блок-схема обратной операции смешения столбцов.

Выше была приведена информация о стандарте AES. Тем не менее, при реализации и попытках применения данного алгоритма имеют место две основные проблемы:

1. Блоки имеют фиксированную длину, в то время как данные могут составлять произвольное число байт.
2. Идентичным блокам исходного текста соответствуют идентичные блоки сокрытого текста.

Первый пункт может быть решен применением принципа дополнения. Второй пункт – выбором режима шифрования, отличного от “Режима простой замены”

Дополнение

Дополнение (англ. *padding*) – принцип предобработки исходного текста, при котором в конец потока данных добавляются незначащие элементы, зависящие от конкретного алгоритма дополнения. Далее будут приведены основные из них.

ANSI X.923

Обозначенный стандарт излагает алгоритм, предполагающий добавление в конец *n-1*-го нулевого байта и одного байта равного n.

Пример: ".. 12 34 **00 00 00 04**"

ISO 10126

Алгоритм стандарта аналогичен предыдущему, за исключением того, что вместо нулевых байт в дополнении следуют случайные данные.
Пример: ".. 12 34 **48 33 38 04**"

PKCS7

PKCS#7 описан в стандарте RFC 5652, где следует, что дополнение должно состоять из n байт, каждый из которых равен n.
Пример: ".. 12 34 **04 04 04 04**"

ISO/IEC 7816-4

Стандарт формулирует, что дополнение должно состоять из одного байта, равного 80_{16} и произвольного числа нулевых байт.
Пример: ".. 12 34 **80 00 00 00**"

Дополнение нулями

Данный принцип подразумевает дополнение нулевых байтов в конец исходного текста.

Не рекомендуется применять подобный подход для общего случая формата данных, так как произвольный формат исходных данных может содержать собственные нули в конце, что сделает информацию с данным дополнением невозможной к достоверной обратной интерпретации.

Режимы шифрования

Режим шифрования в контексте симметричной блочной криптографии подразумевает преобразования данных, помимо основной процедуры.

Выделяют следующие режимы шифрования:

Electronic Codebook (ECB)

Обозначаемый также как режим простой замены, режим ECB осуществляет последовательное применение основной процедуры к блокам данных, формируя сокрытый текст из зашифрованных блоков. Этот подход губителен тем, что идентичным блокам на входе будут соответствовать идентичные блоки на выходе.

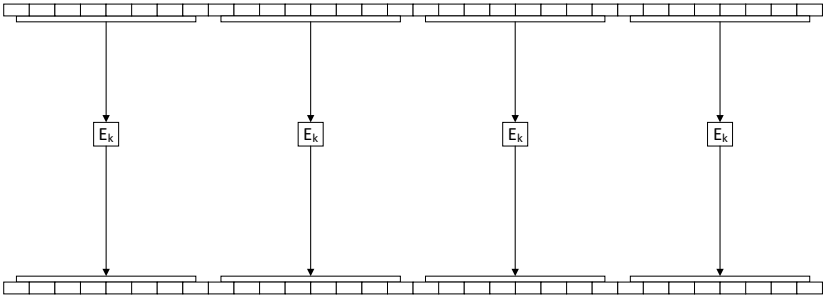


Рисунок 3.11. Схема режима простой замены.

Взглянув на иллюстрацию, можно заметить, что каждый блок обрабатывается вне зависимости от общего потока, таким образом, что обозначалось ранее, повторяющимся блокам на входе будут сопоставлены повторяющиеся блоки на выходе.

Cipher Block Chaining (CBC)

Режим сцепления блоков сокрытого текста предполагает то, что предыдущие результаты шифрования будут влиять на последующие.

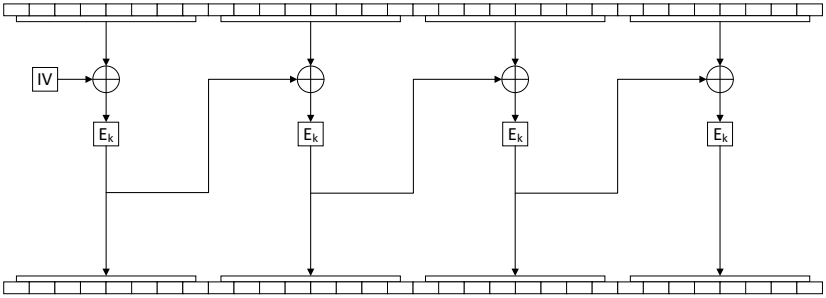


Рисунок 3.12. Схема режима сцепления блоков сокрытого текста.

Взглянув на иллюстрацию, можно заметить, что зашифрованный блок изменяет следующий за ним исходный блок с применением операции XOR.

Данный режим шифрования вводит понятие инициализирующего вектора (IV) – начальный набор байт, определяющий первое воздействие на ход преобразования потока данных.

Инициализирующий вектор выступает в составе общей информации о процессе преобразования данных и необходим, наравне с ключом, для проведения операций шифрования и дешифрования.

Обобщенный алгоритм шифрования представлен далее:
$C_1 = E_k(IV\ xor\ P_1);\ C_i = E_k(C_{i-1}\ xor\ P_i)$ для $i = 2..N.$

Дешифрование, в свою очередь, будет происходить по следующему принципу:
$P'_1 = D_k(C_1)\ xor\ IV;\ P'_i = D_k(C_i)\ xor\ C_{i-1}$, для $i = 2..N.$

Cipher Feedback (CFB)

CFB или режим отдачи по сокрытому тексту. Реализует механизм, при котором основной процедуре шифрования подвергается на первой итерации – инициализирующий вектор, далее – блоки сокрытого текста предыдущей итерации. После основной процедуры шифрования её результат подвергается операции XOR с открытым текстом, результат чего выступает в роли итогового блока сокрытого текста.

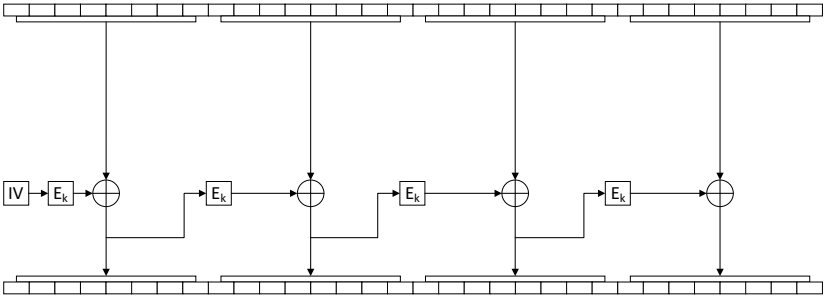


Рисунок 3.13. Схема режима отдачи по сокрытому тексту.

Шифрование в режиме отдачи по сокрытому тексту:
$C_1 = E_k(IV)\ xor\ P_1;\ C_i = E_k(C_{i-1})\ xor\ P_i$ для $i = 2..N$.

Дешифрование данного режима:
$P'_1 = C_1\ xor\ E_k(IV);\ P'_i = C_{i-1}\ xor\ E_k(C_i)$, для $i = 2..N$.

Output Feedback (OFB)

Режим отдачи по выводу. Аналогичен предыдущему режиму, за исключением того, что вместо сокрытого текста очередным блоком в основной процедуре выступает предыдущий результат шифрования. Сокрытый же текст формируется из XOR операции между результатом основной процедуры и исходным текстом.

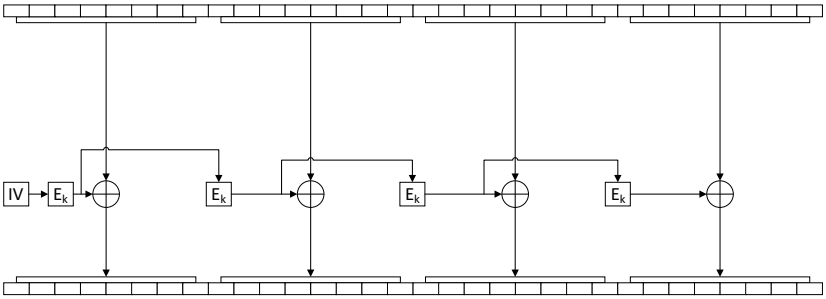


Рисунок 3.14. Схема режима отдачи по выводу

Шифрование в режиме отдачи по выводу:
$C_1 = E_k(IV)\ xor\ P_1;\ C_i = E_k(C_{i-1}\ xor\ P_{i-1})$ для $i = 2..N.$

Аналогично, дешифрование:
$P'_1 = C_1\ xor\ E_k(IV);\ P'_i = C_i\ xor\ E_k(C_{i-1}\ xor\ P_{i-1}),$ для $i = 2..N;$

Имеется ряд других режимов. Тем не менее, в рамках данной работы будут рассмотрены только приведённые.

Альтернативные алгоритмы

Существуют стандарты ГОСТ, также описывающие симметричные алгоритмы шифрования. Самые известные из них – “Магма” разработанный в 1978-м году и относительно новый алгоритм “Кузнечик”.

ГОСТ 28147-89 (“Магма”)

Данный шифр является менее составным, чем AES. Изначально состояние формируется из входного блока путем его разбиения на две составляющие равной протяженности. Итерация алгоритма проводится от значения первой половины, после чего проводится операция XOR результата со второй половиной. Затем происходит запись результата в первую половину, а её предыдущее значение заносится во вторую.

В итерацию алгоритма входит сложение по модулю 2^{32} текущего состояния с ключом, разбитие состояния на 8 блоков по 4 бита, применение нелинейной табличной замены, конкатенация подменённых блоков и сдвиг на 11 бит влево.

Псевдокод алгоритма приведён ниже:

```
Magma(block, key)
begin
a, b = split(block, 2)
rKey = repeat(key, 3) | reverse(key)
for i = 0 to 31 step 1 begin
        t = a xor rKey[i]
        parts = split(t, 8)
        for j = 0 to 7 step 1 begin
                parts[j] = SBoxes[j][parts[j]]
        end
        t = concat(parts)
        t = rotl(t, 11)
        a, b = b xor t, a
end
return concat(a, b)
end
```

Где оператор «|» полагает конкатенацию байт.

Иначе можно проиллюстрировать алгоритм следующим образом:

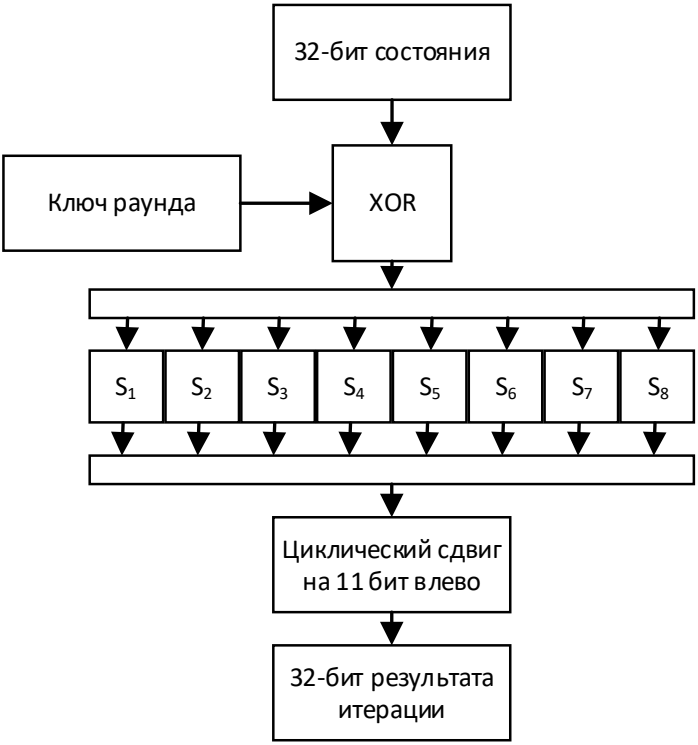


Рисунок 3.15 Алгоритм итерации

$A_{i+1} = B_i \; XOR \; f(A_i, X_i)$

$B_{i+1} = A_i$

Где f – функция итерации, X_i – ключ i-го раунда

Уместно выделить, что ключи раунда образуются на основании базового ключа путем циклического повторения его три раза и затем добавлением обратного порядка ключа с единицей инверсии 32-битным словом, чем и выступает ключ раунда. Иначе говоря,

rKey(X) = $X_1X_2X_3X_4X_5X_6X_7X_8$ $X_1X_2\ldots X_7X_8X_8X_7X_6X_5X_4X_3X_2X_1$

Дешифрование проводится аналогичным образом за вычетом порядка и направления действий и принципа построения ключа раунда.

ГОСТ 34.12-2018 (“Кузнечик”)

Данный алгоритм имеет более составную структуру.

Следует ввести следующие обозначения:

k, a — ключ раунда и 128 битное состояние,

$a = [a_{15}, a_{14}, .., a_1, a_0], a_i$ — i-й байт

$Add_2[k](a) = k \oplus a,$

$N(a) = [S(a_{15}), S(a_{14}), .., S(a_1), S(a_0)]$

$G(a) = [\gamma(a_{15}, a_{14}, .., a_0), a_{15}, a_{14}, .., a_2, a_1]$

$G^{-1}(a) = [a_{14}, a_{13}, a_{12}, .., a_0, \gamma(a_{14}, .., a_1, a_0)], G^{-1}[G(a)] = a$

$H(a) = G^{16}(a) = G^{15}[G(a)], = ..$

$F[k](a_1, a_0) = (H \circ N \circ Add_2[k](a_1) \oplus a_0, a_1)$

Здесь и далее, $A \circ B(x) = A[B(x)]. C \circ (A \circ B) = C \circ A \circ B$

$$\gamma(a_{15}, .., a_0) = 148 * a_{15} + 32 * a_{14} + 133 * a_{13} + 16 * a_{12} + 194 * a_{11} + 192 * a_{10} + a_9 + \\ + 251 * a_8 + a_7 + 192 * a_6 + 194 * a_5 + 16 * a_4 + 133 * a_3 + 32 * a_2 + 148 * a_1 + a_0$$

При этом операции проводятся в поле GF(2^8) с порождающим полиномом $x^8 + x^7 + x^6 + x + 1$

S(x) = SBox[x]

SBox= {
252, 238, 221, 17, 207, 110, 49, 22, 251, 196, 250, 218, 35, 197, 4, 77, 233, 119, 240, 219, 147,
46, 153, 186, 23, 54, 241, 187, 20, 205, 95, 193, 249, 24, 101, 90, 226, 92, 239, 33, 129, 28, 60, 66,
139, 1, 142, 79, 5, 132, 2, 174, 227, 106, 143, 160, 6, 11,237, 152, 127, 212, 211,31,235, 52, 44, 81,
234, 200, 72, 171,242, 42, 104, 162, 253, 58, 206, 204, 181, 112, 14, 86, 8, 12, 118, 18, 191, 114, 19,
71, 156, 183, 93, 135, 21,161, 150, 41, 16, 123, 154, 199, 243, 145, 120, 111, 157, 158, 178, 177, 50,
117, 25, 61,255, 53, 138, 126, 109, 84, 198, 128, 195, 189, 13, 87, 223, 245, 36, 169, 62, 168, 67,
201,215, 121,214, 246, 124, 34, 185, 3, 224, 15, 236, 222, 122, 148, 176, 188, 220, 232, 40, 80, 78,
51, 10, 74, 167, 151, 96, 115, 30, 0, 98, 68, 26, 184, 56, 130, 100, 159, 38, 65, 173, 69, 70, 146, 39,
94, 85, 47, 140, 163, 165, 125, 105, 213, 149, 59, 7, 88, 179, 64, 134, 172, 29, 247, 48, 55, 107, 228,
136, 217, 231, 137, 225, 27, 131,73, 76, 63, 248, 254, 141,83, 170, 144, 202, 216, 133, 97, 32, 113,
103, 164, 45, 43, 9, 91,203, 155, 37, 208, 190, 229, 108, 82, 89, 166, 116, 210, 230, 244, 180, 192,
209, 102, 175, 194, 57, 75, 99, 182
}

Далее приведён алгоритм генерации ключа:

$C_i = H(i), i = 1,2,..,32$

$K_1 = (k_{255}, .., k_{128}).\ K_2 = (k_{127}, .., k_0)$

$(K_{2i+1}, K_{2i+2}) = F\left[C_{8(i-1)+8}\right] \circ .. \circ F\left[C_{8(i-1)+1}\right](K_{2i-1}, K_{2i}), i = 1,2,3,4$

Далее указаны алгоритмы шифрования и дешифрования:
$E(a) = Add_2[K_{10}] \circ H \circ N \circ Add_2[K_9] . . \circ H \circ N \circ Add_2[K_1](a)$
$D(a) = Add_2[K_1] \circ N^{-1} \circ H^{-1} \circ Add_2[K_2] \circ . . \circ N^{-1} \circ H^{-1} \circ Add_2[K_{10}](a)$

В течение десяти лет с момента публикации у данного шифра не было выявлено явных уязвимостей. Тем не менее, на первых 9-и из 10-и итераций осуществляется операция H, включающая 16 операций γ, которые в свою очередь включают 16 операций умножения в поле Галуа. Несмотря на то, что действие может быть оптимизировано с помощью таблиц линейных подстановок, всё же число таких операций на одной итерации составляет 256. Для AES аналогичная операция MixColumns включает всего 16 умножений. Аналогичная картина присутствует в пунктах сдвигов. Операция H совмещает в себе, помимо линейных преобразований, и действия диффузии, сдвигая отдельные байты в 128-битном состоянии. Действие H включает 256 операций перемещения данных, в то время как аналогичная операция ShiftRows задействует всего 12 перемещений на одной итерации.

Содержание работы

1. *Реализовать две подпрограммы – дополнения байт и обратную к этому операцию, по одному из приведённых алгоритмов, в соответствии с вариантом:

Вариант	Алгоритм дополнения
1, 5, ..	ANSI X.923
2, 6, ..	ISO 10126
3, 7, ..	PKCS7
4, 8, ..	ISO/IEC 7816-4

2. Реализовать стандарт симметричного шифрования AES. Реализация подразумевает наличие функции шифрования как отдельных блоков фиксированного размера, так и произвольного массива данных.
   Для операции обработки произвольных данных подразумевается включение в алгоритм процедуры дополнения при шифровании и применении обратной операции после дешифрования.

3. *Реализовать один из режимов шифрования, отличных от ECB, включить в общий конвейер обработки данных возможность использования заданного режима.

Вариант	Режим шифрования
1, 4, ..	CBC
2, 5, ..	CFB
3, ..	OFB

4. Реализовать графический интерфейс, предоставляющий следующий функционал
   4.1. Установка ключа в текстовом и шестнадцатеричном виде.
   4.2. *Установка инициализирующего вектора в обозначенных форматах.
   4.3. Указание обрабатываемых данных в обозначенных форматах, из поля формы или из файла.
   4.4. Проведение операции шифрования.
   4.5. Проведение операции дешифрования.

Контрольные вопросы

1. Конечные поля.
   1.1. Определение конечного поля.
   1.2. Элементы конечного поля.
   1.3. Понятие порождающего полинома.
   1.4. Принципы взятия значения по модулю в поле Галуа.
2. Представление данных в конечном поле Галуа $GF(2^8)$.
3. Описание общего алгоритма в рамках стандарта AES.
   3.1. Порядок шифрования;
   3.2. Порядок дешифрования.
4. Описание отдельных процедур в рамках стандарта AES.
   4.1. Key Expansion;
   4.2. AddRoundKey
   4.3. SubBytes;
   4.4. ShiftRows;
   4.5. MixColumns;
   4.6. Обратные версии 4.3-4.5.
5. Описание основных принципов дополнения данных.
   5.1. ANSI X.923;
   5.2. ISO 10126;
   5.3. PKCS7;
   5.4. ISO/IEC 7816-4.
6. Описание основных режимов шифрования.
   6.1. ECB;
   6.2. CBC;
   6.3. CFB;
   6.4. OFB.
7. ГОСТ 28147-89
8. ГОСТ 34.12-2018

Лабораторная работа №4.

ГЕНЕРАЦИЯ ПРОСТЫХ ЧИСЕЛ. АЛГОРИТМЫ ПРОВЕРКИ ПРИНАДЛЕЖНОСТИ ПРОСТЫМ ЧИСЛАМ

Цель выполнения работы заключена в знакомстве с принципами получения простых чисел высокой разрядности, имеющими приложение в рамках ряда алгоритмов асимметричного шифрования. Помимо этого, работа подразумевает знакомство с перечнем вероятностных и детерминированных алгоритмов определения принадлежности числа ко множеству простых.

Краткие теоретические сведения

Используемая терминология

Простое число – число, имеющее в качестве делителей лишь два числа – единицу и самого себя. Каждое целое число представимо единственным образом в виде уникального произведения простых чисел.

$\forall x \in Z: \exists! \, \alpha = \{\alpha_2, \alpha_3, \alpha_5, \alpha_7, ..\}: x = 2^{\alpha_2} * 3^{\alpha_3} * 5^{\alpha_5} * ..$

Пример: $x = 12, a = \{2,1,0,0, ..\}. x = \ 2^2 * 3^1$

Наибольший общий делитель. НОД$(a, b) \ = \ max(k : \ k|a \wedge k|b)$,
где x|y обозначает то, что x делит y без остатка.

Детерминированный алгоритм. Алгоритм проверки принадлежности числа к простым, дающий точный результат.

Вероятностный алгоритм. Алгоритм проверки принадлежности числа к простым, дающий ответ лишь с той или иной вероятностью.

Генерация случайных чисел

Для работы ряда криптографических алгоритмов требуется применение генерации случайных чисел. Случайные числа при формировании ключевых данных позволяют получить тот результат, который бы соответствовал отдельным критериям приемлемости ключа, а также был бы малопредсказуемым и устойчив от атак грубой силы. Случайные числа, генерируемые тем или иным алгоритмом, должны соответствовать ряду условий.

Из таких условий выделяют:

1. **Однородность**. Данное условие подразумевает то, что вероятность появления каждого числа из диапазона генерируемых значений была близка к равномерной.
2. **Независимость**. Условие независимости подразумевает, что очередное число генерируемой последовательности не должно быть представлено функциональной зависимостью от предыдущих чисел.

Данные условия позволяют сказать, что алгоритм генерации является безопасным и применим в криптографических алгоритмах.

В действительной практике, можно наблюдать отступление от данных положений. В таком случае уместно упрощение второго условия до того, что следующее число нельзя было предугадать на основании предыдущего.

Генерация простых чисел высокой разрядности

Простые числа высокой разрядности находят применение в таких алгоритмах как RSA, в протоколе Диффи-Хелмана и схеме Эль-Гамаля.

Принцип генерации простых чисел основан на том, что их распределение сопоставимо с натуральным логарифмом от верхней границы значений. Вероятность принадлежности случайного числа n к простым числам примерно равна $\frac{1}{ln(n)}$. На основании этого, генерация простых чисел связана с взятием случайных чисел до тех пор, пока не будет найдено простое число. Проверка принадлежности числа

простым для случая высокой разрядности чаще всего основана на вероятностных алгоритмах.

Вероятностные алгоритмы

Вероятностные алгоритмы дают ответ на принадлежность числа простым лишь с некоторой вероятностью, тем не менее, они работают быстрее детерминированных алгоритмов.

Допустимо выделить следующие вероятностные алгоритмы проверки принадлежности простым числам:

1. Метод случайного общего делителя.
2. Метод Ферма.
3. Метод Рабина — Миллера.
4. Метод Соловея — Штрассена

Математические основы вероятностных алгоритмов.

Для дальнейшего рассмотрения, требуется ввести понятия об арифметике вычетов, функции Эйлера, теоремах Ферма и Эйлера, о символах Лежандра и Якоби.

Арифметика вычетов

Любое натуральное число m образует кольцо вычетов относительно себя.

Кольцо вычетов представляет множество из m элементов – нуля и m-1 первых натуральных чисел.

В кольце соблюдается большинство арифметических соответствий, за исключением того, что результат каждой операции в кольце представляет остаток от деления на m аналогичной операции.
Например: m = 3. Тогда кольцом выступит следующее множество:
$\mathbb{Z}_3 = \{0,1,2\}$

*, $\mathbb{N}$	0	1	2
0	0	0	0
1	0	1	2
2	0	2	4

*, $\mathbb{Z}_3$	0	1	2
0	0	0	0
1	0	1	2
2	0	2	(2*2) mod 3 = 1

Любое целое число может быть переведено в аналогичный ему элемент кольца вычетов. Для этого требуется представить число в виде
$x = k*m + b$,
где $k \in \mathbb{Z}$, $b \in \mathbb{Z}_n$.

В таком случае говорят, что число x сравнимо по модулю m с числом b.

$x \equiv b \ (mod\ m)$
Где $x = k * m + b$

Сравнимые по модулю m числа образуют класс вычетов по данному модулю. Все данные числа дают общий остаток при делении на m.

Нотация n mod m обозначает взятие остатка от деления n на m.

Функция Эйлера
Функция Эйлера $\varphi(x)$ – представляет функциональное соответствие числа натурального ряда количеству взаимно простых с ним чисел, меньших его.

Для простых чисел, $\varphi(p) = p - 1$.

Для взаимно простых a,b, $\varphi(ab) = \varphi(a) \cdot \varphi(b)$

Для степени целого числа $\varphi(a^n) = a^{n-1} \cdot \varphi(a)$

Для числа 12, значение функции Эйлера будет равно 4-м:
$\varphi(12) = \varphi(4 * 3) = \varphi(4) * \varphi(3) = [2^{2-1} * \varphi(2)] * 2 = 2 * 1 * 2 = 4$
Взаимно простыми числами с 12-ью будут выступать 1,5,7,11.

Данная функция применяется в ряде случаев ввиду её свойств, одно из которых сформулировано в теореме Эйлера.

Теорема Эйлера
Для взаимно простых чисел a и m, соблюдается следующее выражение:
$a^{\varphi(m)} = 1 \ (mod\ m)$

Теорема Ферма

Частный случай теоремы Эйлера для простого m:
$a^{p-1} \equiv 1\ (mod\ p)$, где $p \in P$

Из данного факта следует в частности, что:
$a^x \equiv a^{x\ mod\ (p-1)}\ (mod\ p)$

Данные положения позволяют применять теорему Ферма в прикладных аспектах, в частности – в алгоритмах асимметричной криптографии. Второе свойство в совокупности с коммутативностью показателей возведения в степень позволяет строить криптосистемы с парами ключей один из которых может быть публичным.

Примером может служить a = 3, p = 41:
$3^{41-1}\ mod\ 41 = 3^{40} mod\ 41 = 12157665459056928801\ mod\ 41 = 1$

Квадратичный вычет

Число a называется квадратичным вычетом по модулю p, если существует число *x*, ставящее следующее выражение в соответствие:
$x^2 \equiv a\ (mod\ p)$

Примитивными случаями выступают квадраты, меньшие p. Примером иных случаев может служить любой квадрат, взятый по модулю p:
a = 4 для p = 12, x = 4;
a = 1 для p = 48, x = 7.

Символ Лежандра

Символ Лежандра – функция от двух переменных: a – произвольного целого числа и p – простого числа, большего, чем два.
Обозначается как $\left(\frac{a}{p}\right)$ и формулируется следующим образом:

- $\left(\frac{a}{p}\right) = 0$, если a делится на p
- $\left(\frac{a}{p}\right) = 1$, если a не делится на p, но является квадратичным вычетом по модулю p.
- $\left(\frac{a}{p}\right) = -1$ в иных случаях

Примером могут служить следующие случаи: $\left(\frac{6}{3}\right) = 0, \left(\frac{1}{11}\right) = 1, \left(\frac{2}{11}\right) = -1$

Символ Якоби

Обобщение символа Лежандра для произвольного нечетного p.

Определяется следующим образом: при $b = p_1 p_2 .. p_n$, где $p_i \in P$, $\left(\frac{a}{b}\right) = \left(\frac{a}{p_1}\right)\left(\frac{a}{p_2}\right)..\left(\frac{a}{p_n}\right)$. В частности, из этого следует, так как число 1 не является простым, что $\forall a \in \mathbb{N}$: $\left(\frac{a}{1}\right) = 1$.

Алгоритм вычисления символа Якоби:

1. Для $a < 0$: $\left(\frac{a}{n}\right) = \left(\frac{-a}{n}\right) * (-1)^{\frac{n-1}{2}}$.
2. Для $a = 2k$: $\left(\frac{a}{n}\right) = \left(\frac{a/2}{n}\right) * (-1)^{\frac{n^2-1}{8}}$
3. При $a = 1$, $\left(\frac{a}{n}\right) = 1$
4. При $a < n$, $\left(\frac{a}{n}\right) = \left(\frac{n}{a}\right) * (-1)^{\frac{a-1}{2}*\frac{n-1}{2}}$
5. Иначе, $\left(\frac{a}{n}\right) = \left(\frac{a \bmod n}{n}\right)$.

Метод случайного общего делителя

Для проверки принадлежности числа n простым числам, уместно использовать алгоритм, заключенный в следующем:

Выделяется число d: $2 \leq d < n$.

Если d имеет наибольший общий делитель с n, отличный от единицы, вернуть флаг составного числа, иначе перейти к следующей итерации.

Эффективность данного метода сопряжена со значением функции Эйлера.

Вероятность того, что составное число n пройдет этот тест, равна $\frac{\varphi(n)}{n}$.

Мат. ожидание функции Эйлера составляет порядка $\frac{6}{10} n * O(\log n)$.

Таким образом, вероятность того, что составное число пройдет данный тест, составляет примерно 0.6, не пройдет – 0.4.

Для достижения вероятности p, требуется провести k тестирований, где $k = \left\lceil -\log_{\frac{5}{3}}(1-p) \right\rceil$

Метод Ферма

В обозначенном методе используется аппарат упомянутой ранее теоремы Ферма. Принцип алгоритма заключен в следующем:

1. Выбирается $a \in \mathbb{Z}: 2 \leq a \leq n-1$.
2. Если тождество $a^{n-1} \equiv 1\ (mod\ n)$ не соблюдается, вернуть результат "n – составное", иначе вернуться к шагу 1.
3. Если по прошествии k итераций тождество соблюдалось, вернуть результат "n – вероятно простое"

Существуют числа Кармайкла, которые, при выборе любых взаимно простых с ними чисел a дают тождество в выражении теоремы Ферма. Тем не менее, при выборе произвольных целых чисел a, алгоритм Ферма распознаёт данные числа как составные, прим.: n = 561, a = 9.

Метод Рабина—Миллера

Данный алгоритм также опирается на малую теорему Ферма, тем не менее, отличным от стандартного определения образом.

Он опирается на следующее следствие теоремы:
Пусть $n \in P, n-1 = 2^s d,$ где d – нечетно. Тогда для любого $a \in \mathbb{Z}_n$ выполняется по меньшей мере одно из условий:

1. $a^d \equiv 1\ (mod\ n)$
2. Найдется r: r < s, соответствующее тождеству $a^{2^r d} \equiv -1\ (mod\ n)$

Следствие позволяет построить следующий алгоритм:

1. Разложить $n-1$ на $2^s \cdot d$, где d – нечетно.
2. Выбрать $a \in \mathbb{Z}: 2 \leq a \leq n-2$
3. $x = a^d\ mod\ n$
4. Если $x = n-1$ или $x = 1$, перейти к шагу 2.
5. Возвести x в квадрат по модулю n.
6. При $x = n-1$, перейти к шагу 2
7. Если $x^2 < n-1$, перейти к шагу 5, иначе вернуть "составное".
8. Если по прошествии k итераций второго шага результат не был получен, вернуть "вероятно простое"

Метод Соловея—Штрассена

Данный метод использует аппарат квадратичных вычетов и формулируется следующим образом:

1. Выбрать $a \in \mathbb{Z}: 2 \leq a \leq n-1$
2. При $\text{НОД}(a,n) \neq 1$, вернуть "составное"
3. При $a^{\frac{n-1}{2}} \equiv \left(\frac{a}{n}\right) \ (mod\ n)$, перейти к шагу 1, иначе вернуть "составное"
4. Если по прошествии *k* итераций шага 1 результат не был получен, вернуть "вероятно простое".

Далее представлены усреднённые вероятности ошибок для каждого из алгоритмов. Из них можно сделать вывод об их эффективности.

N_{max}\k	1	2	3	4	5	6
10	0.014231	0.002106	0.000305	0.000040	0.000009	0.000000
10^2	0.027952	0.004717	0.001099	0.000371	0.000106	0.000052
10^3	0.013528	0.001971	0.000521	0.000206	0.000083	0.000046
10^4	0.005089	0.000779	0.000394	0.000255	0.000185	0.000117
10^5	0.001619	0.000240	0.000096	0.000069	0.000059	0.000037
10^6	0.000434	0.000060	0.000033	0.000021	0.000018	0.000017

Таблица 4.1. Вероятности для алгоритма Ферма.

N_{max}\k	1	2	3	4	5	6
10	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
10^2	0.009568	0.001122	0.000151	0.000023	0.000005	0.000001
10^3	0.005728	0.000433	0.000069	0.000004	0.000003	0.000000
10^4	0.001924	0.000094	0.000013	0.000001	0.000000	0.000000
10^5	0.000503	0.000024	0.000003	0.000001	0.000000	0.000000
10^6	0.000136	0.000002	0.000002	0.000000	0.000000	0.000000

Таблица 4.2. Вероятности для алгоритма Рабина—Миллера.

N_{max}\k	1	2	3	4	5	6
10	0.10571	0.03626	0.01499	0.00678	0.00320	0.00159
10^2	0.06034	0.01220	0.00339	0.00114	0.00046	0.00019
10^3	0.03900	0.00671	0.00172	0.00043	0.00011	0.00004
10^4	0.02270	0.00348	0.00116	0.00028	0.00008	0.00001
10^5	0.01353	0.00322	0.00094	0.00018	0.00006	0.00001
10^6	0.01090	0.00254	0.00071	0.00018	0.00004	0.00001

Таблица 4.3. Вероятности для алгоритма Соловея—Штрассена.

Детерминированные алгоритмы проверки принадлежности простым числам

Все рассмотренные алгоритмы являлись вероятностными, не давая точного результата. В ситуации, когда факт принадлежности заданного числа множеству простых чисел должен быть точным, уместно применять детерминированные алгоритмы.

Среди них допустимо выделить следующие:

1. Метод перебора делителей.
2. Метод решета Эратосфена.
3. Метод Агравала—Каяла—Саксены.

Метод перебора делителей

Метод заключен в переборе чисел от *2* до $\sqrt{n}$ включительно. Если хоть одно из чисел делит n, то n – составное, иначе – n принадлежит простым числам.

Данный алгоритм имеет слабость, выраженную в числе необходимых операций взятия остатка от деления: верхняя грань числа операций составляет $\frac{n_b}{2}$. Для 64-ёх битного числа, число операций составит 2^{32}. При частоте 2^{32} операций в секунду, выполнение проверки займет одну секунду.
Уже для 128-ми битного значения, проверка займёт 2^{64} секунд, или 136 лет.

Метод решета Эратосфена

Алгоритм решета заключен в формировании булева массива и задания его значений следующим образом:
1. $sieve_0$ = $sieve_1$ = Ложь, остальные значения – Истина.
2. Для k от 2 до n:
3. Если $sieve_k$ = Ложь, перейти к следующему k.
4 Иначе для j от 2k до n с шагом k:
5 $sieve_j$ = Ложь
6. Вернуть sieve

Преимущества алгоритма решета заключены в том, что достаточно один раз сформировать массив за O(n) операций, после чего можно апеллировать к нему за единичную операцию. Если задача включает проверку большого массива чисел, не превышающих по значению некоторого n, то алгоритм решета будет демонстрировать результаты, превышающие эффективность вероятностных алгоритмов – они будут работать за O(m*log(n)), алгоритм решета – за O(n+m). Тем не менее, алгоритм всё же не подходит для значений высокой разрядности, как по времени выполнения, так и по выделяемой памяти.

Метод Агравала—Каяла—Саксены

Данный алгоритм примечателен тем, что является логарифмическим – асимптотика операций для его выполнения оценивается как $O(\log^6 n)$.

Сам алгоритм выглядит следующим образом:

1. При $\exists a, b \in \mathbb{N}, a, b > 1: n = a^b$, вернуть "составное".
2. Выбрать наименьшее r, соответствующее $ord_r(n) > log^2 n$
3. При $1 < НОД(a, n) < n$ для $a \leq r$, вернуть "составное".
4. При $n \leq r$, вернуть "простое".
5. Для каждого a от 1 до $\left\lfloor \sqrt{\varphi(r)} log_2(n) \right\rfloor$:
При $(x + a)^n \neq x^n + a \ (mod\ x^r - 1, n)$, вернуть "составное"
6. Вернуть "простое".

Несмотря на эффективность алгоритма, для чисел достаточно высокой разрядности, он всё ещё неприменим и не может быть поставлен в один ряд с вероятностными алгоритмами.

Генерация простого числа

Во многих прикладных задачах криптографии, возникает необходимость генерации простого числа заданной разрядности. Далее приведён один из вариантов алгоритма генерации.

1. Сгенерировать случайное n-битное число p.
2. Установить старший и младший биты, равными единице. Старший бит позволит убедиться в необходимой разрядности, младший – в нечётности.
3. Осуществить проверку делимости на малые простые числа: 3, 5, 7, 11, ..
4. Выполнить тест Рабина—Миллера с кратностью не менее 6-и.

Если число не прошло проверку в пунктах 3 или 4, перейти к пункту 1.
Иначе вернуть число p.

Проверка на отдельные простые числа отсекает составные с вероятностью $\frac{1}{p}$. Для 3-ёх, 5-ти и 7-и вероятность составит 54%. При дальнейшей проверке до 256-ти, будут отсечены 80% чисел.

Содержание работы

1. Реализовать подпрограмму проверки принадлежности числа простым числам по рассмотренным детерминированным алгоритмам.
   1.1. Метод перебора делителей.
   1.2. Метод решета.
2. Реализовать подпрограмму для проверки принадлежности числа простым числам по рассмотренным вероятностным алгоритмам:
   2.1. Метод случайного общего делителя.
   2.2. Метод Ферма.
   2.3. Метод Рабина — Миллера.
   2.4. *Метод Соловея — Штрассена
3. С использованием одного из вероятностных методов, реализовать алгоритм генерации простого числа заданной разрядности. Пользователь должен иметь возможность задавать количество бит в числе и параметр числа итераций k.
   Должна быть возможность передать число бит, превышающих 128.
4. Реализовать программный интерфейс, позволяющий применить модули, описанные выше.

Контрольные вопросы

1. Принцип работы следующих вероятностных алгоритмов:
   1.1. Метод Ферма.
   1.2. Метод Рабина—Миллера.
2. Принцип работы следующих детерминированных алгоритмов:
   2.1. Метод перебора делителей.
   2.2. Метод решета.
3. Понятие чисел Кармайкла.
4. Понятие псевдопростого числа.
5. Принцип генерации простых чисел.
6. Определение функции Эйлера.
7. Понятие о символе Лежандра.
8. Понятие о символе Якоби.

Лабораторная работа №5.

АЛГОРИТМ ОБМЕНА КЛЮЧАМИ ДИФФИ-ХЕЛЛМАНА. СХЕМА ЭЛЬ-ГАМАЛЯ.

Цель работы заключена в изучении алгоритма обмена ключами Диффи-Хеллмана и последующей реализации данного алгоритма.

Краткие теоретические сведения

Криптографический протокол – набор правил, описывающий последовательность действий для двух и более участников (единиц, узлов) обмена информацией. Цель задания правил – решение задачи защиты информации методами криптографии.

Одна из важнейших проблем криптографии – передача сообщения по незащищенному каналу связи. Симметричное шифрование накладывает ограничение на то, каким образом должны передаваться ключи. Незащищенный канал может служить каналом передачи зашифрованных симметричным алгоритмом данных. Тем не менее ключ симметричного алгоритма должен быть распределён между участниками обмена данных таким образом, чтобы третьи по отношению к участникам обмена данными лица не получили доступ к зашифрованной информации. В связи с этим уместно обозначить задачу распределения ключей.

Задача распределения ключей может быть решена применением асимметричных алгоритмов шифрования.

Математические основы асимметричной криптографии

Для введения понятия асимметричной криптографии, следует ознакомиться с математическим базисом, составляющим понятия о первообразном корне, коммутативности показателей при возведении в степень, о приведённой системе вычетов.

Приведённая система вычетов

Для кольца вычетов по модулю m существует подмножество её элементов, формирующее мультипликативную группу относительно операции умножения.

Группа – замкнутое относительно определённой на ней операции множество, имеющее нейтральный элемент и для каждого из остальных элементов – обратные.

Замкнутость подразумевает, что для любых $a, b \in M: (a \circ b) \in M$
Нейтральный элемент I выражен следующим отношением
$\forall a \in M: (a \circ I) = a$
Обратный элемент определяем следующим высказыванием:
$\forall a \in M: \exists b \in M: b \circ a = I$

Примером группы может выступить множество целых чисел и операция сложения. I = 0, обратный элемент для x: $-x$. Для любой пары целых чисел a и b, c = a + b также целое, следовательно группа замкнута.

Также как группу уместно выделить рациональные числа и операцию умножения. I = 1, обратный элемент для x: $\frac{1}{x}$. Аналогично, число c = a*b для любых рациональных a,b также рационально.

Для мультипликативной группы кольца вычетов операцией выступает умножение по модулю, нейтральным элементом выступает единица.

Теперь имеет место выделить, что элементами мультипликативной группы кольца вычетов могут служить лишь элементы, взаимно простые с основанием кольца. Это обуславливается тем, что, если найдётся $x \in Z_n$, где $\text{НОД}(x, n) \neq 1$, то $\nexists y: x * y = 1$,
так как $\forall k: k * n + x \equiv x \ (mod\ \text{НОД}[x, n])$,
точно также как $\forall m: \text{НОД}[x * m, n] \geq \text{НОД}[x, n]$

Таким образом, если x не является взаимно простым с основанием кольца n, обратный элемент y не может быть найден. Это обусловлено тем, что умножение x на множитель m не может понизить наибольший

общий делитель x и n, ровно также, как вычитание кратных n из того или иного числа, так как $\text{НОД}(k * n, n) = n$.

Таким образом, мультипликативные преобразования над числом, не являющимся взаимно простым с основанием кольца n, не смогут сформировать единицу. Обратного элемента для такого числа быть не может.

Из этого положения выводится понятие о приведённой системе вычетов.

Приведённая система вычетов – подмножество элементов кольца вычетов, взаимно простых с основанием кольца. Обозначенная система позволяет определить на ней операцию умножения по модулю, вместе с данной операцией система образует группу, на которой соблюдаются обозначенные ранее свойства.

Первообразный корень

Для приведённой системы вычетов допустимо выделить следующие свойства: система может быть упорядочена относительно степеней ряда элементов. Для $n = 7, Z_7 = \{1,2,3,4,5,6\}$,
при этом, $3^0 = 1, 3^1 = 3, 3^2 = 2, 3^3 = 6, 3^4 = 4, 3^5 = 5, 3^6 = 1$.
Последовательность (1,3,2,6,4,5) является степенями тройки по модулю семи. Уместно заметить, что, вследствие того, что степени вида $6 * k + b$ будут давать то же число, что и степень b, уместно также отметить, что взаимно простые с шестью значения степеней тройки также будут образовывать последовательность, аналогичную обозначенной. Для пяти: (1,5,4,6,2,3).

При этом нетрудно заметить, что степени, не являющиеся взаимно простыми с шестью, будут давать иную картину ввиду того, что минимальная степень для таких чисел, при которых будет достигаться нейтральный элемент, будет сопоставима с n/k. Это обусловлено тем, что возведение числа t^y в степень x умножает степень y на данное значение x. Таким образом, степень становится равна $x*y$. Если $xy = 0\ (mod\ 6)$, результатом возведения в степень выступит единица.

Обозначенные положения обобщимы на произвольные основания колец. Тогда число элементов в группе будет составлять $\varphi(n)$. В свою очередь число, образующее группу относительно операции возведения в степень будет обозначаемо как первообразный корень.

Число первообразных корней в свою очередь будет составлять $\varphi(\varphi(n))$. Данное положение исходит из того, что в последовательности $(1, \alpha, \alpha^2, .., \alpha^{\varphi(n)-1})$ будет расположено столько взаимно простых с $\varphi(n)$ степеней α, чему равно значение функции Эйлера от числа элементов, тем же образом чем определяется данная функция.

Для простых n, число элементов в группе составляет $n-1$, число же первообразных корней составляет $\varphi(n-1)$.

Получение первообразного корня по модулю n

Первообразные корни существуют для оснований $n = 2, 4, p^\alpha, 2p^\alpha$ и отсутствуют для других значений основания[1].

Первым шагом следует проверка модуля принадлежности обозначенным случаям.

Вторым шагом выступит выбор произвольного *x*.

Далее требуется провести проверку того, что число является первообразным корнем.

Для этого уместно выделить два критерия:

1. Наибольший общий делитель *x* и *n* равен единице.
2. $\forall p \in P, d \in \mathbb{N}, d = \frac{\varphi(n)}{p} : x^d \not\equiv 1 \ (mod\ n)$

Пример первообразного корня для $n = 41$: $g = 22$, т. к. $22^{\frac{40}{5}} \equiv 37 \ (mod\ n)$, $22^{\frac{40}{2}} \equiv 40 \ (mod\ n)$, в свою очередь $g' = 5$ не является корнем, так как $5^{\frac{40}{2}} \equiv 1 \ (mod\ 41)$

Основной проблемой данного подхода выступает необходимость поиска простых делителей значения $\varphi(n)$.

---

[1] Ю.В. Нестеренко. Теория чисел. // Глава 7. // https://mmmf.msu.ru/lect/nesterenko/mainnth.pdf

Решением данной проблемы в задачах, где используются первообразные корни, может выступить выбор $n = 2 * p + 1, p \in P$. Для подобного основания справедливо соотношение $\varphi(n) = 2 * p$.
В таком случае пропадает необходимость поиска всех делителей $\varphi(n)$, так как они известны изначально, поиск первообразного корня производится за время порядка $O(log^2 N_{bit})$.

Коммутативность показателей при возведении в степень
Из свойств возведения в степень следует, что $(x^a)^b = (x^b)^a = x^{ab}$.

Например, $x=2$, $a=2$, $b=3$: $x^a=4$, $x^b=8$, $4^3=8^2=64=x^{ab}$

Асимметричная криптография

Асимметричная криптография появилась в связи с вызовами, которые были обусловлены понятием незащищенного канала. Проблема атаки “Человек посередине” представляется достаточно острой. Асимметричная криптография позволяет решить данную проблему.

Асимметричная криптография является подмножеством криптографии, сфокусированном на алгоритмах, включающих в рассмотрение понятие открытого и закрытого ключей.

Для асимметричных алгоритмов, открытый ключ может быть известен всем участникам беседы, и факт получения открытого ключа третьим лицам не позволит осуществить компрометацию данных.

Закрытый ключ в свою очередь должен быть известен только отдельным участникам беседы, как правило одному, и не должен передаваться в открытом виде по незащищенным каналам.

Задача дискретного логарифмирования

Одной из открытых математических проблем является задача о дискретном логарифме – поиске основания α по заданному $x^\alpha \ mod \ m$

Наилучшие результаты на данный момент – алгоритмы, занимающие $O(\sqrt{m})$ операций. Таким образом, возведение числа в произвольную степень является условно необратимой для достаточно высоких значений m.

Протокол обмена ключами Диффи—Хеллмана

Коммутативность показателей при возведении в степень может быть применена в приведённой системе вычетов для реализации асимметричной криптографии.

Для известного первообразного корня g и модуля n уместно сформировать два сокрытых ключа – a и b. Один участник беседы возводит g в степень a и отправляет $A = g^a$ второму частнику. Второй участник, в свою очередь, проводит аналогичные действия со степенью b – отправляет $B = g^b$ первому собеседнику. Далее участники беседы возводят полученное им числа в свои частные ключи, получая общее значение: $A^b = B^a = g^{ab} = X$.

Пример данных: n = 23, g = 20, a = 14, A = 4, b = 12, B = 3, X = 4.

Даже при условии того, что канал обмена данными является незащищенным, согласно обозначенной ранее проблеме дискретного логарифма, числа A и B, передаваемые по общему каналу, в совокупности с основанием n и первообразным корнем g, не позволят скомпрометировать обмен данными.

Алгоритм обмена ключами выразим на псевдокоде следующим образом:

Предварительная процедура
Выбрать простые p, n = 2*p+1
Выбрать первообразный корень g по модулю n
Распределить g, n между абонентами.

Первый абонент
Выбрать взаимно простое с $\varphi(n)$ значение a
Вычислить $A = g^a \bmod n$
Отправить A второму абоненту
Ожидать поступления B от второго абонента
Вычислить $K = B^a \bmod n$

Второй абонент
Выбрать взаимно простое с $\varphi(n)$ значение b
Вычислить B = g^b mod n
Отправить B первому абоненту
Ожидать поступления A от первого абонента
Вычислить K = A^b mod n

После выполнения своих процедур каждым из абонентов, им обоим будет доступен ключ K, который, в свою очередь, не будет доступен потенциальным третьим лицам, прослушивающим канал связи.

Идея применения неразрешимости задачи дискретного логарифмирования в асимметричной криптографии получила развитие в схеме Эль-Гамаля.

Следует отметить факт того, что при выборе в качестве основания системы число g, не являющееся первообразным корнем по модулю n, может возникнуть ситуация, при которой мультипликативная группа имела бы низкое число элементов. Это может возникнуть при выборе числа, которое является степенью k первообразного корня, при этом k делит n-1. В таком случае число элементов в группе будет равно (n-1)/k. Для достаточно высоких k, этот факт может сыграть существенную роль в нарушении безопасности криптосистемы.

Схема Эль-Гамаля

Схема Эль-Гамаля является продолжением идей, предложенных в рамках протокола Диффи—Хеллмана. Данная схема позволяет передавать в защищенном виде по открытому каналу произвольные сообщения. В данной схеме имеет место применение представления об обратимости операции умножения в кольце и наличии обратных элементов.

Схема заключена в следующем: выбираются простые $p, n = 2 * p + 1$
Выбирается первообразный корень g по модулю n, выбирается произвольное число s: $1 < s < n - 1$, вычисляется $S = g^s \, mod \, n$.
Открытым ключом выступает кортеж (S, g, n), закрытым – s.

Для того, чтобы зашифровать сообщение M по данным параметрам требуется произвести следующие операции:
Выбрать сессионный ключ k: $1 < k < n-1$.
Вычислить $u = g^k \ mod \ n$ и $v = S^k M \ mod \ n$. Кортеж (u, v) выступит в качестве зашифрованных данных.
Дешифрование будет основано на факте того, что
$v = (g^s)^k M = g^{sk} M = u^s M$
Из этого следует, что возведя u в степень закрытого ключа s, можно получить $t = vM^{-1}$, тогда $t^{-1} = v^{-1}M$, следовательно $M = (u^s)^{-1} * v$
Здесь важно подчеркнуть, что x^{-1} в кольце вычетов заданных параметров требует возведения *x* в степень *n-2*. Тем не менее, это не соблюдается для произвольных оснований кольца, и, в общем случае, требует применения расширенного алгоритма Евклида.

На псевдокоде алгоритм схемы Эль-Гамаля будет выглядеть следующим образом:

Предварительная процедура
Выбрать простые *p*, *n = 2*p+1*
Выбрать первообразный корень g по модулю *n*
Выбрать *s:* $1 < s < n-1$
Вычислить $S = g^s \ mod \ n$

Процедура шифрования
Вход: сообщение M в виде *l*-битного числа
Выход: результат шифрования
Выбрать сессионный ключ *k:* $1 < k < n-1$
Вычислить $u = g^k \ mod \ n$
Вычислить $v = S^k M \ mod \ n$
Вернуть кортеж *(u,v)*

Процедура дешифрования
Вход: кортеж зашифрованных данных *(u,v)*
Выход: исходное сообщение в виде *l*-битного числа
Вычислить $t = u^s \ mod \ n$
Вычислить $M' = t^{-1} v \ mod \ n$
Вернуть *M'*

Применение данной схемы для обмена данными включает в себя выполнения предварительной процедуры на принимающей стороне, последующую за этим передачу кортежа *(S, g, n)* отправляющей стороне. Отправляющая сторона выполняет процедуру шифрования, передавая кортеж *(u, v)* принимающей стороне. Та в свою очередь осуществляет процедуру дешифрования.

Содержание работы

1. Реализовать программный модуль, реализующий следующие операции, связанные с первообразными корнями:
   1.1. Проверка того, что заданный произвольный модуль может иметь первообразные корни
   1.2. Получение первых *k* первообразных корней для произвольного заданного числа
   1.3. Генерация простого n вида n = 2p + 1, где p – также простое.
   1.4. Получение первообразного корня для заданного n вида 2p+1
2. Реализовать модуль, моделирующий обмен ключами по протоколу Диффи—Хеллмана.
3. Реализовать модуль, реализующий процедуры схемы Эль-Гамаля, приведённые выше.
4. Реализовать программный интерфейс, включающий в себя следующие пункты:
   4.1. Страница первообразных корней. Страница должна включать:
      4.1.1. Поля задания значений n и k
      4.1.2. Возможность получения первых k первообразных корней по модулю n с отображением затраченного времени.
   4.2. Страница протокола Диффи—Хеллмана. Страница должна включать:
      4.2.1. Поле задания числа бит в основании n.
      4.2.2. Возможность моделирования протокола с заданной точностью, с отображением шагов обмена.
   4.3. *Страница схемы Эль-Гамаля. Страница должна включать
      4.3.1. Поля задания числа бит в основании n, самого n, ключа s, первообразного корня g.
      4.3.2. Возможность генерации обозначенных s, g, n по числу бит.
      4.3.3. Возможность получения зашифрованного текста в шестнадцатеричном виде по заданным ключам и сообщению.
      4.3.4. Возможность дешифрования шестнадцатиричного представления зашифрованного текста по заданным ключам с выводом сообщения в одном из представлений.

Контрольные вопросы

1. Понятие асимметричной криптографии. Отличие от симметричного подмножества дисциплины. Понятие открытого и закрытого ключа.
2. Понятие приведённой системы вычетов.
3. Понятие первообразного корня.
4. Свойство коммутативности показателей при возведении в степень.
5. Принцип работы протокола Диффи—Хелмана.
6. Принцип работы схемы Эль-Гамаля.

Лабораторная работа №6.

КОМБИНИРОВАННЫЕ АЛГОРИТМЫ ШИФРОВАНИЯ

Цель работы заключается в рассмотрении вопросов, возникающих при обмене данными по незащищенному каналу, знакомстве с комбинацией асимметричных и симметричных алгоритмов при передаче информации, в реализации приложения, моделирующего данный подход.

Краткие теоретические сведения

Симметричные алгоритмы, как было указано ранее, имеют главную слабость – несмотря на то, что нельзя восстановить исходный текст, опираясь исключительно на сами зашифрованные данные, тем не менее, при компрометации ключа шифрования, вся криптосистема теряет смысл. Исходя из этих предпосылок, при использовании симметричных алгоритмов для обмена данными, их ключ должен передаваться тем образом, чтобы эта передача не влекла за собой его компрометацию. Эта проблема может быть решена с использованием средств, рассмотренных ранее, обозначенных как асимметричные алгоритмы.

В таком случае возникнет вопрос о том, почему нельзя использовать асимметричные алгоритмы для шифрования самих данных. Ответом на данный вопрос выступает принципиальное отличие в быстродействии – симметричные алгоритмы могут быть выполнены за $O(n_{bit})$ операций, в то время как рассмотренные ранее алгоритмы асимметричной криптографии работают за $O(n_{bit}^3)$ – используется операция возведения в степень разрядности n_{bit}, что выполнимо за $O(n_{bit})$ операций умножения. Операция умножения в свою очередь выполняется за $O(n_{bit})$ операций сложения, сложение за $O(n_{bit})$ атомарных операций, что, впрочем, зависит от реализации арифметики высокой разрядности. Так или иначе, при использовании асимметричных алгоритмов возникает проблема быстродействия.

Данная проблема разрешима с применением комбинированных алгоритмов.

Один из основоположников асимметричной криптографии Уитфилд Диффи отмечал, что необходимость предварительного распределения ключей симметричных алгоритмов заведомо саботирует процесс обмена данными. Данный подход нарушает принцип Керкгоффса, так как распределённый заранее ключ может быть скомпрометирован и тем самым будет скомпрометирована вся криптосистема.

Второй проблемой выступает достоверность в том, что все участники обмена данными действительно является теми лицами, которыми представляются. Данная проблема также именуется проблемой аутентификации.

Уитфилд Диффи и Матрин Хеллман достигли значительных результатов, предложив решение обеим задачам, радикально отличное от всего того, что было раньше.

Ключ симметричного шифрования обозначим как *секретный ключ*. Пару ключей асимметричного шифрования обозначим как *открытый* и *закрытый* ключи. Закрытый ключ должен быть сокрытым и доступен лишь отдельным лицам обмена данными, открытый ключ – общеизвестен. Закрытый ключ обозначим как KR, открытый ключ – как KU.

Диффи и Хеллман описывают требования, которым должен соответствовать алгоритм шифрования с открытым ключом:

1. Возможность создания пары (KU, KR)
2. При наличии открытого ключа и сообщения M, возможность формирования сокрытого сообщения $C = E_{KU}(M)$
3. При наличии закрытого ключа и сокрытого сообщения C, возможность восстановить исходное сообщение $M = D_{KR}(C)$
4. Невозможность, исходя лишь из знаний об открытом ключе, определить закрытый ключ
5. Невозможность получения сообщения M лишь по открытому ключу KU и сокрытому сообщению C.

6. Коммутативность применения операций шифрования и дешифрования – $E_{KU}[D_{KR}(M)] = D_{KR}[E_{KU}(M)] = M$. Данное свойство, впрочем, не является действительно необходимым для обозначенных криптосистем, и существуют функционирующие системы, несоответствующие ему.

Односторонней функцией называется функция, которая позволяет получить значение на выходе, не позволяющее простым образом восстановить исходно заданный аргумент, в то время как сама функция вычисляется достаточно просто.

Здесь под "простым" понимается характеристик полиномиальное от числа разрядов время. Если число бит во входном значении равно n, то функция должна выполняться за $O(n^a)$ действий, где a является константой.

Восстановление значения должно занимать принципиально больше действий. На данный момент следует полагать, что временная оценка должна составлять $O(2^n)$.

Резюмируя, можно выделить то, что асимметричные алгоритмы применимы при работе с незащищенным каналом, симметричные – с крупными блоками информации. Для первых характерно свойство наличия двух ключей, для вторых – быстродействие.

Общая схема асимметричных алгоритмов

Шифрование с открытым ключом, одна из реализаций которого была рассмотрена ранее, в общем виде представляет следующий порядок действий:

1. Принимающая сторона генерирует пару ключей (KU, KR)
2. Принимающая сторона отправляет посылающей стороне открытый ключ.
3. Посылающая сторона применяет операцию шифрования к своему сообщению, получая сокрытые данные $C = E_{KU}(M)$, которые она отправляет принимающей стороне.
4. Принимающая сторона дешифрует сообщение, используя закрытый ключ $M' = D_{KR}(C)$

Алгоритм асимметричной криптографии RSA

Алгоритм RSA является последователем протокола Диффи—Хеллмана, изложенным годом позже обозначенного алгоритма. В 1977-м году Рон Ривест, Ади Шамир и Лен Адлеман опубликовали работы, описывающие данный алгоритм. Алгоритм использует факт того, что факторизация произвольного числа не является полиномиальной от его разрядности. Наилучшие результаты демонстрируют ρ-алгоритм Полларда и метод эллиптических кривых, тем не менее, ни один из них не позволяет решать обозначенную задачу для достаточно высокого числа бит.

Алгоритм RSA заключается в следующем:

Первым пунктом выступает генерация пары ключей.

1. Осуществляется выбор двух простых чисел p, q
2. Вычисляется $n = pq$
3. Вычисляется значение функции Эйлера для n, равное $\varphi(n) = [\text{т. к. НОД}(p, q) = 1]\ \varphi(p) * \varphi(q) = (p-1)(q-1)$
4. Выбирается число $e, 1 < e < \varphi(n), \text{НОД}[\varphi(n), e] = 1$
5. Вычисляется $d = e^{-1}\ mod\ \varphi(n)$
6. Открытым ключом выступит кортеж (e, n), закрытым – (d, n).

После этого шифрование проводится по формуле $C = M^e\ mod\ n$.
Дешифрование – по формуле $M' = C^d\ mod\ n$.

RSA предполагает возможность блочного шифрования. В таком случае формулы идентичны до индексов в шифруемых и дешифруемых данных – $C_k = M_k^e\ mod\ n,\ M'_k = C_k^d\ mod\ n$. Само собой, данные положения справедливы при предположении использования режима ECB, для иных режимов порядок преобразования модифицируется в соответствии с конкретными положениями.

Здесь имеет место замечание того факта, что здесь значение $\varphi(\varphi(n))$ заведомо неизвестно, в связи с чем обратный элемент e^{-1} не может быть вычислен подходом, применённым в предыдущей работе. Данная задача здесь может быть решена расширенным алгоритмом Евклида.

Расширенный алгоритм Евклида

Стандартный алгоритм Евклида позволяет вычислить наибольший общий делитель на основании справедливости соотношения

$НОД(A,B) = НОД(B, A\ mod\ B)$ для $A, B > 1$

Данные положения расширяются задачей вычисления значений x, y таких, что $A * x + B * y = НОД(A,B)$. В частности, это позволит получить $x = d$ при $A = e,\ B = \varphi(n)$.

Обозначенная задача разрешима следующим образом:

Будут использоваться три последовательности – r, t и h

На первом шаге определяются первые элементы:
r_0 = A, r_1 = B;
t_0 = 1, t_1 = 0;
h_0 = 0, h_1 = 1

Дальнейшие элементы определяются следующим образом:

$d_i = r_{i-1}\ div\ r_i\ ;\ r_{i+1} = r_{i-1}\ mod\ r_i;$

$t_{i+1} = t_{i-1} - d_i t_i; s_{i+1} = s_{i-1} - d_i s_i$

При достижении $r_{k+1} = 0$, вычисления прекращаются. В таком случае уместно говорить, что $s_k = x$ в выражении, приведенном ранее. t_k при этом будет выступать в роли y.

Получив таким образом x уместно говорить, что $x\ mod\ \varphi(n) = d$

Операция осуществима без выделения дополнительной памяти с использованием лишь 12-ти переменных. Псевдокод приведён ниже:
Входные параметры:
A – число, поиск обратного элемента, для которого осуществляется
M – основание кольца, относительно которого ведётся поиск
Выходные параметры: A^{-1} – обратный элемент относительно A
$r_0 = A$; $r_1 = M$
$t_0 = 1$; $t_1 = 0$
$s_0 = 0$; $s_1 = 1$
Повторять до тех пор, пока r_1 не примет значение 0
    $d = r_0 \text{ div } r_1$; $r = r_0 \bmod r_1$
    $s = s_0 - d*s_1$; $t = t_0 - d*t_1$
    $r_0, r_1 = r_1, r$
    $s_0, s_1 = s_1, s$
    $t_0, t_1 = t_1, t$
Конец цикла
Вернуть $s_0 \bmod M$

Комбинированные алгоритмы шифрования

Использование симметричных алгоритмов для шифрования данных и асимметричных для шифрования ключей позволит, сохраняя производительность первых алгоритмов, передавать данные по незащищенному каналу без угроз компрометации.

Псевдокод комбинированного подхода выглядит следующим образом:
k – ключ симметричного алгоритма, ES_k, DS_k – операция шифрования и дешифрования симметричного алгоритма, e, d – открытый и закрытый ключи асимметричного алгоритма, EP_e, DP_d – операция шифрования и дешифрования асимметричного алгоритма, P, C – исходные и сокрытые данные

На отправляющей стороне	*На принимающей стороне*
Получить e	Отправить e
$C = ES_k(P)$	Получить (C, S)
$S = EP_e(k)$	$k = DP_d(S)$
Отправить (C, S)	$P = DS_k(C)$

Содержание работы

1. Реализовать модуль, предоставляющий возможность применять алгоритм RSA.
2. Реализовать или использовать ранее реализованный модуль, предоставляющий возможность применять алгоритм AES.
3. Реализовать модуль комбинированного алгоритма шифрования данных.
4. Построить приложение с графическим пользовательским интерфейсом, предоставляющим доступ к реализованному функционалу. В приложение должен входить следующий функционал:
   - 4.1. Возможность задания числа бит в значениях p и q
   - 4.2. Возможность генерации значений p и q или задание их из файла
   - 4.3. Возможность сохранения сгенерированных ключей для алгоритма RSA
   - 4.4. Генерация симметричного ключа для алгоритма AES.
   - 4.5. Возможность моделирования работы комбинированного алгоритма, включающего:
     - 4.5.1. Шифрование исходного текста симметричным алгоритмом
     - 4.5.2. Сохранение зашифрованного текста в отдельном файле
     - 4.5.3. Шифрование ключа асимметричным алгоритмом, сохранение результата в отдельном файле
     - 4.5.4. Дешифрование ключа.
     - 4.5.5. На основе дешифрованного значения – дешифрование сокрытого текста пункта 4.5.1 симметричным алгоритмом.
     - 4.5.6. Сохранение результата пункта 4.5.5 в отдельном файле.

Контрольные вопросы

1. Проблема распределения симметричного ключа.
2. Причины применения симметричных алгоритмов.
3. Требования к асимметричным алгоритмам.
4. Принцип комбинированного шифрования.
5. Алгоритм RSA.
6. Расширенный алгоритм Евклида.

Лабораторная работа №7

АЛГОРИТМЫ КОНТРОЛЬНЫХ СУММ. ЭЛЕКТРОННАЯ ПОДПИСЬ.

Цель работы заключается в знакомстве студента с алгоритмом получения контрольной суммы SHA-256. В изучении понятия электронной подписи и реализации алгоритмов формирования и проверки подлинности сформированных подписей.

Краткие теоретические сведения

В задачах криптографии часто возникает потребность в преобразовании данных таким образом, чтобы результат преобразования не давал возможности получить исходные данные. При этом получать сами исходные данные из результата нет необходимости, и результат представляет собственную ценность. Такие алгоритмы обозначаются как контрольные суммы.

Однонаправленные функции контрольных сумм

Однонаправленная функция $Prfsm(M)$ применяется к сообщению произвольной разрядности M и возвращается контрольная сумма фиксированной разрядности $s = Prfsm(M)$.

Достаточно просто составить функцию, суммирующую данные произвольной протяженности, тем не менее, криптографическая функция контрольной суммы должна соответствовать дополнительным требованиям, приведённым ниже:

1. При знании M, должна быть возможность легко вычислить $s = Prfsm(M)$, где s составляет фиксированную длину, M – произвольную.
2. При знании только $s = Prfsm(M)$, не должно быть возможности легко вычислить M.
3. При знании M и $s = Prfsm(M)$, не должно быть возможности легко вычислить M', соответствующее условию $Prfsm(M') = s, M' \neq M$.

Первое требование позволяет использовать алгоритм в структурах наподобие хеш-таблиц.

Второе и третье требования определяют функцию, применимую в криптографических системах.

Атака «дней рождения»

Под атакой «дни рождения» подразумевается попытка найти коллизию алгоритма контрольной суммы. Под коллизией понимается наличие M', для которого $Prfsm(M) = Prfsm(M')$.

Основанием атаки служит свойство совпадающих равно распределённых значений. Для таких значений, получение очередного уникального случая после k итераций составляет $\frac{n-k}{n}$, так как осталось только $n - k$ уникальных вариантов из общего числа n возможных.

Число операций, необходимых для обнаружения коллизии с вероятностью, превышающей 0.5, составляет $O(\sqrt{n})$

Далее приведён протокол, демонстрирующий, что если третий пункт требований к криптографическому алгоритму контрольной суммы не соблюдается, этот факт может быть использован для саботажа надежности криптосистемы.

1. Сторона А готовит две версии контракта – одну, выгодную для стороны Б, вторую выгодную для А.

2. Пользователь А вносит незаметные изменения в оба документа. Такими изменениями могут служить добавление пробелов в конце переноса строк, добавление переносов строк перед концом страницы и так далее. Таким образом при наличии 32 позиций, подверженных изменениям можно получить 2^{32} вариантов документа.

3. Пользователь А вычисляет контрольные суммы для сформированных вариаций исходных документов. При совпадении сумм для вариаций двух изначальных документов, эти две версии фиксируются

4. Пользователь А отправляет вариант документа, выгодный для стороны Б этой стороне.

5. Пользователь А получает подпись документа, являющуюся валидной также и для выгодной для А версии.

6. Пользователь А может апеллировать к факту наличия подписи как к согласию стороны Б с выгодной для А версии контракта.

Данный пример является основанием сказать о том, что криптографический алгоритм контрольной суммы должен иметь достаточную разрядность, давать на выходе примерно равновозможные значения и иметь свойство лавинного эффекта. Это свойство полагает, что малые изменения в аргументе влекут сильные изменения на выходе.

Сжатие данных в алгоритмах контрольной суммы

Понятие сжатия включает в себя следующее соотношение:

$s_i = f\ (M_i, s_{i-1})$

Данное положение позволяет работать с данными произвольной протяженности. Данные разбиваются на блоки фиксированной длины. Каждый следующий блок модифицирует состояние результата, тем не менее, обрабатывается отдельно.

MD-усиление

Для того, чтобы усложнить криптоанализ алгоритмов контрольных сумм, часто применяется принцип, обозначаемый MD-усилением. Данный принцип полагает включение числа байт сообщения в обрабатываемый поток данных.

Алгоритм контрольной суммы SHA-256

SHA-256 представляет криптографический алгоритм контрольной суммы. Данный алгоритм был принят в качестве стандарта в 2001-м году.

Данный алгоритм подразумевает обработку сообщений, длиной до 2^{64} бит ввиду алгоритма дополнения, сопряженного с действием самого SHA-256. Разрядность одного блока составляет 512 бит. Разрядность выходного значения составляет 256 бит.

Алгоритм последовательно обрабатывает блоки данных, модифицируя результат. При обработке очередного блока, текущее состояние принимается в расчет, что устраняет проблему идентичности перестановок блоков.

Первым шагом в алгоритме выступает инициализация состояния результата. Результат представлен в виде восьми 32-разрядных переменных. Изначальное заполнение этих переменных псевдослучайно, основано на двоичном представлении квадратных корней первых восьми простых чисел.

```
h₀ := 0x6A09E667
h₁ := 0xBB67AE85
h₂ := 0x3C6EF372
h₃ := 0xA54FF53A
h₄ := 0x510E527F
h₅ := 0x9B05688C
h₆ := 0x1F83D9AB
h₇ := 0x5BE0CD19
```

Вторым шагом выступает инициализация массива констант. Данные приведены в приложении 5.

Следующим шагом выступит дополнение сообщения. К сообщению добавляется один единичный бит и k нулевых битов. k выбирается таким образом, чтобы выражение $L + 1 + k$ при делении на 512 давало остаток 448, где L – число бит в исходном сообщении. После этого сообщение дополняется двоичным представлением L от старшего бита к младшему.

После обозначенных преобразований, сообщение разбивается на блоки разрядности 512 бит, после чего каждый блок обрабатывается изложенным далее образом.

Для преобразований в рамках обработки отдельного блока, уместно ввести следующие обозначения:

$ROTR^n(x)$ – циклический сдвиг значения x вправо на n бит;
$SHR^n(x)$ – логический сдвиг значения x вправо на n бит;
$\Sigma_0(x) = ROTR^2(x) \oplus ROTR^{13}(x) \oplus ROTR^{22}(x)$
$\Sigma_1(x) = ROTR^6(x) \oplus ROTR^{11}(x) \oplus ROTR^{25}(x)$
$s_0(x) = ROTR^7(x) \oplus ROTR^{18}(x) \oplus SHR^3(x)$
$s_1(x) = ROTR^{17}(x) \oplus ROTR^{19}(x) \oplus SHR^{10}(x)$
$Ch(x,y,z) = (x \,\&\, y) \oplus (\neg x \,\&\, z)$
$Ma(x,y,z) = (x \,\&\, y) \oplus (x \,\&\, z) \oplus (y \,\&\, z)$

Предварительная обработка блока включает формирование массива 32-битных значений следующим образом:
1. w[0..15] = M_{batch}[0..15]
2. Для i от 16 до 63:
3. S0 = s_0(w[i-15]); S1 = s_1(w[i-2])
4. w[i] = w[i-16] + S0 + w[i-7] + S1

Далее 32-битные переменные a,b,..,h заполняются значениями h_0,h_1,..,h_7

После данных действий осуществляется основной цикл:
1. Для i от 0 до 63:
2. t1 = h+Σ_1(e)+Ch(e,f,g)+k_i+w_i
3. t2 = Σ_0(a) + Ma(a,b,c)
4. h = g
5. g= f
6. f = e
5. e = d + t1
7. d = c
8. c= b
9. b = a
10. a = t1 + t2

После выполнения основного цикла, осуществляется модификация состояния:

$h_0 = h_0 + a$

$h_1 = h_1 + b$

$h_2 = h_2 + c$

..

$h_6 = h_6 + g$

$h_7 = h_7 + h$

После этого обработка блока завершается.

После завершения обработки всех блоков, формируется контрольная сумма, как результат конкатенации переменных h_0-h_7 от первой к последней.

Электронная подпись

Симметричные криптосистемы не являются применимыми в вопросах формирования электронной подписи, так как удостоверение в подлинности подписи предполагает наличие ключа, позволяющего точно также сформировать подпись. Эта проблема может быть решена с использованием асимметричных криптосистем.

Формируется контрольная сумма документа, после чего сумма подписывается закрытым ключом. Для проверки подлинности допустимо применить публичный ключ к подписи и сопоставить результат применения с вычисленной контрольной суммой документа. Если значения совпадают – подпись подлинна.

На электронную подпись накладывают следующие ограничения:

1. Уникальность.
2. Отсутствие возможности отказа от выполненной подписи.
3. Возможность верификации любым заинтересованным лицом.
4. Возможность простым образом осуществлять формирование и верификацию подписей
5. Невозможность подделать подпись.
6. Возможность обработки больших объемов данных.

Для формирования электронной подписи уместно применение одного из следующих алгоритмов:

1. RSA
2. Digital Signature Algorithm, DSA.
3. Алгоритм Эль-Гамаля для электронной подписи
4. Алгоритм стандарта ГОСТ Р 34.10-94.
5. Elliptic Curves Digital Signature Algorithm или ECDSA – алгоритм формирования цифровой подписи на основании арифметики эллиптических кривых.
6. Алгоритм Эль-Гамаля на эллиптических кривых.

Электронная подпись на основе алгоритма RSA

Алгоритм получения электронной подписи на основе алгоритма RSA аналогичен алгоритму шифрования данных на основе этого алгоритма.

Далее приведён псевдокод операций формирования и проверки подлинности подписи:

Формирование подписи

Входные данные: сообщение M, закрытый ключ e, основание n
Выходные данные: подпись S
prfsm := Prfsm(M)
S := $prfsm^e$ mod n
Вернуть S

Выше и далее Prfsm – функция получения контрольной суммы.

Проверка подлинности

Входные данные: сообщение M, открытый ключ d, основание n, подпись S
Выходные данные: результат проверки
prfsm := Prfsm(M)
S' := S^d mod n
Если prfsm = S', вернуть "подпись подлинна"
Иначе, вернуть "подпись некорректна"

Характерным отличием RSA в режиме подписи является то, что зашифровать данные может только фигурант подписи, а дешифровать – любое заинтересованное лицо, что точно аналогично до обратного порядка в режиме шифрования

Электронная подпись на основе алгоритма Эль-Гамаля

Рассмотренная ранее схема Эль-Гамаля применима в задаче построения цифровой подписи. Далее приведён псевдокод операций формирования и заверения подписи.

Предварительная процедура

Выбрать простые p, $n = 2*p+1$
Выбрать первообразный корень g по модулю n
Выбрать x: $1 < x < n-1$
Вычислить $y = g^x \bmod n$

Формирование подписи

Входные данные: простое число n, g – первообразный корень из n, закрытый ключ x, сообщение M
Выходные данные (r, s) – подпись сообщения
prfsm = Prfsm(M)
Выбрать k, 1 < k < n-1, НОД(k, n-1) = 1
$r = g^k \bmod n$
$s = (prfsm – x*r)k^{-1} \pmod{n-1}$
Вернуть (r, s)

Проверка подписи

Входные данные: простое число n, g – первообразный корень из n, сообщение M, открытый ключ y, подпись (r, s)
Выходные данные: результат проверки
prfsm = Prfsm(M)
$S = y^r r^s \pmod n$
$G = g^{prfsm} \pmod n$
Если G = S, вернуть вернуть "подпись подлинна"
Иначе, вернуть "подпись некорректна"

Схема Эль-Гамаля в режиме подписи использует подход, отличающийся более характерным образом. Факт того, что для схемы Эль-Гамаля не соблюдается 6-ой пункт требований к асимметричным алгоритмам, обуславливает невозможность адаптировать алгоритм для подписи простой переменой порядка действий. Алгоритм использует свойства операции возведения в степень и умножения. Параметр s принимает в расчёт контрольную сумму, добавляя нагрузку скрытых параметров для устранения угрозы компрометации. После операций, слагаемые сокращаются, оставляя только корень в степени суммы.

Прикладная составляющая вопроса электронной подписи

Электронная подпись в прикладном разрезе может использоваться для заверения произвольных электронных документов. Подпись может также использоваться в других механизмах аутентификации и подтверждать, что авторизуемое лицо действительно обладает правами доступа к тому или иному ресурсу.

Виды электронной подписи

В соответствии с законодательством Российской Федерации, выделяют три класса электронных подписей. Далее приведена выдержка из Статьи 5 Федерального закона от 06.04.2011 N 63-ФЗ (ред. от 21.04.2025) "Об электронной подписи"

1. Простая электронная подпись. Задана в произвольном виде – кодом, паролем или иной формой, позволяя идентифицировать лицо, осуществляющее подписание.

2. Усиленная неквалифицированная электронная подпись – подпись, которая характеризуется следующими свойствами:

2.1. Получена в результате криптографического преобразования информации с использованием ключа электронной подписи

2.2. Позволяет определить лицо, подписавшее документ.

2.3. Позволяет обнаружить факт внесения изменений в электронный документ после момента подписания.

2.4. Создается с использованием средств электронной подписи.

3. Усиленная квалифицированная электронная подпись – соответствует всем признакам усиленной неквалифицированной электронной подписи и следующим дополнительным признакам:

3.1. Ключ проверки подписи указан в квалифицированном сертификате;

3.2. Для создания и проверки электронной подписи используются средства электронной подписи, имеющие подтверждение соответствия требованиям, установленным в соответствии с настоящим Федеральным законом.

Российские альтернативы в сфере контрольных сумм

Стандарт ГОСТ 34.11-2018 описывает алгоритм получения контрольной суммы, описанный далее:

Входные данные: инициализирующий вектор IV, сообщение M.

Этап 1.

Шаг 1.1. h = IV

Шаг 1.2. N = $0^{512} \in V_{512}$

Шаг 1.3. S = $0^{512} \in V_{512}$

Шаг 1.4. Перейти к этапу 2.

Этап 2.

Шаг 2.1. Проверить соответствие |M| < 512

При положительном результате, перейти к этапу 3.

Иначе к шагу 2.2.

Шаг 2.2. Получить вектор m, равный последним 512 бит

Шаг 2.3. h = $g_N(h, m)$

Шаг 2.4. N = $Vec_{512}(Int_{512}(N) \boxplus 512)$

Шаг 2.5. S = $Vec_{512}\big(Int_{512}(S) \boxplus Int_{512}(m)\big)$

Шаг 2.6. Отсечь от M последние 512 бит.

Шаг 2.7. Перейти к шагу 2.1.

Этап 3.

Шаг 3.1. m = $0^{511-|M|}||1||M$

Шаг 3.2. h = $g_N(h, m)$

Шаг 3.3. N = $Vec_{512}(Int_{512}(N) \boxplus |M|)$

Шаг 3.4. S = $Vec_{512}\big(Int_{512}(S) \boxplus Int_{512}(m)\big)$

Шаг 3.5. h = $g_0(h, N)$

Шаг 3.6. h = $\begin{cases} g_0(h, S) \text{ для версии 512 бит} \\ \text{первые 256 бит значения выше, для версии 256 бит.} \end{cases}$,

Шаг 3.7. Конец работы алгоритма

Подробное описание каждого из составляющих алгоритма можно найти в упомянутом стандарте.

Содержание работы

1. Разработать модуль, предоставляющий возможность получения контрольной суммы по алгоритму SHA-256.
2. Разработать модуль, предоставляющий возможность формировать цифровую подпись и проверять её достоверность. Должны быть поддерживаемы алгоритмы RSA и Эль-Гамаля.
3. Реализовать приложение с графическим пользовательским интерфейсом, в функционал которого должны входить следующие составляющие:
   3.1. Возможность получения контрольной суммы по указываемым данным. Должна быть возможность передавать данные как в шестнадцатеричном виде, так и в виде ASCII-строки.
   3.2. Возможность генерировать ключи для электронной подписи. Требуется реализовать возможность задания разрядности ключей. Ключи должны быть сохранены в файлы по указанным путям после генерации
   3.3. Возможность формирования электронной подписи для указанных данных и ключа. Данные и ключ задаются в виде файлов. Подпись также сохраняется в файл и отображается в шестнадцатеричном виде на экране.
   3.4. Возможность проверки заданной электронной подписи по данным и открытому ключу. Подпись, данные и ключ передаются в виде файлов.
4. Осуществить тестирование Разработанных модулей и приложения. Тестирование производится по данным, приведённым в приложении 6.

Контрольные вопросы

1. Понятие контрольной суммы. Общая характеристика криптографических алгоритмов контрольных сумм.
2. Атака "Дней рождения".
3. Понятие сжатия данных при формировании контрольной суммы. Понятие MD-усиления.
4. Принцип работы алгоритма SHA-256.
5. Понятие цифровой подписи.
6. Задача формирования цифровой подписи. Причины применения асимметричной криптографии.
7. Принцип работы алгоритма RSA для цифровой подписи.
8. Принцип работы схемы Эль-Гамаля для цифровой подписи.

Лабораторная работа №8

КРИПТОГРАФИЯ ЭЛЛИПТИЧЕСКИХ КРИВЫХ

Цель работы заключена в освоении методов криптографии с использованием аппарата эллиптических кривых и в реализации данных алгоритмов.

Краткие теоретические сведения

Теория эллиптических кривых имеет прикладное применение в сфере криптографии. Алгебраические свойства полей на эллиптических кривых предоставляют аппарат, применимый в асимметричной криптографии.

Эллиптические кривые

Эллиптические кривые – класс кривых третьего порядка, задаваемых уравнением третьей степени следующего вида:

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$$

В криптографии в свою очередь нашёл применение подкласс данных кривых, задаваемых уравнением следующего вида:

$$y^2 = ax^3 + bx + c$$

Данное уравнение задаёт множество точек (x, y), для которых определены операции сложения и умножения, образуя в совокупности с ними эллиптическую группу $E(a, b)$.

Операция сложения определяется следующим образом:

Рассматривается прямая, построенная от двух слагаемых точек A, B. Если прямая пересекает кривую в третьей точке C, то результатом сложения выступит симметричная ей относительно оси Ox точка C'. (Рисунок 8.1)

Если прямая является касательной к кривой в одной из точек A, B, в таком случае результатом сложения выступит точка, противоположная точке касания.

Если прямая проходит параллельно оси Oy, результатом выступит бесконечно удалённая точка O. Данная точка также является нейтральным элементом группы $\forall P \in E: P + O = P$.

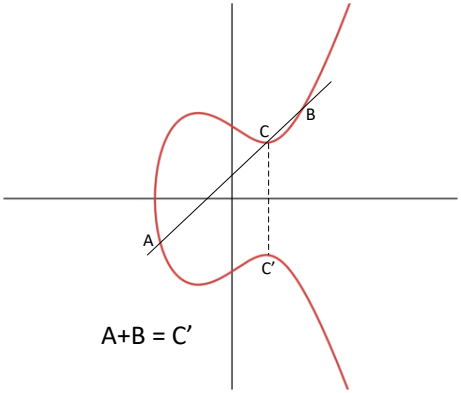


Рисунок 8.1. Случай пересечения в трёх точках.

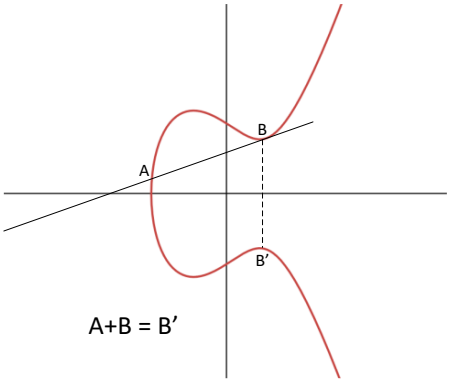


Рисунок 8.2. Случай пересечения в одной точке и касании

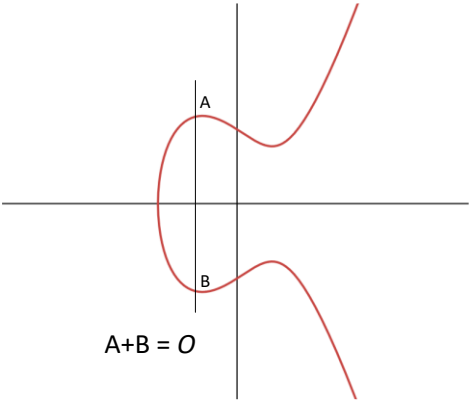


Рисунок 8.3. Сложение противоположных точек.

Таким образом, для эллиптической группы определены следующие элементы и операции:

1. Нейтральный элемент – бесконечно удаленная точка O.
2. Элементы (x, y), при подстановке в уравнение кривой дающие равенство.
3. Противоположные элементы $A = (x, y)$, $A' = (x. -y)$, при сложении дающие результатом нейтральный элемент: $A + A' = O$. (см. Рисунок 8.3)
4. Общая операция сложения трёх точек прямой, пересекающей кривую, результатом которой будет бесконечно удаленная точка.
5. Следствие из пункта 4, позволяющее определить бинарную операцию сложения как точку, противоположную третьей точке пересечения кривой с прямой по двум слагаемым точкам (см. Рисунок 8.1)
6. Второе следствие из пункта 4 для случая касательной, где $A + B = A'$ либо $A + B = B'$, при касательной в A и в B соответственно (см. Рисунок 8.2)

Эллиптическая группа может быть определена на конечном поле F_q.

Эллиптической группой на конечном поле $E_q(a,b)$ выступает множество точек (x,y), для которых уравнение $y^2 \equiv x^3 + ax + b\ (mod\ q)$ обращается в тождество. При этом должно соблюдаться соотношение $4a^3 + 27b^2 \not\equiv 0\ (mod\ q)$.

Для таких групп также будет существовать нейтральный элемент O и обратные элементы $(x,y), (x,z)$, где $z \equiv -y\ (mod\ q)$.

В свою очередь операция сложения в данных группах может быть определена аналитически.

Для $P_1(x_1, y_1)$ и $P_2(x_2, y_2)$, $P_1 + P_2 = P_3$

При этом:
$P_3 = (x_3, y_3), x_3 \equiv \lambda^2 - x_1 - x_2\ (mod\ q); y_3 \equiv (x_1 - x_3)\lambda - y_1 (mod\ q)$

В свою очередь

$$\lambda \equiv \begin{cases} \dfrac{y_2 - y_1}{x_2 - x_1}\ (mod\ q), & \text{при } x_2 \not\equiv x_1\ (mod\ q) \\ \dfrac{3x_1^2 - a}{2y_1}\ (mod\ q), & \text{при } x_2 \equiv x_1\ (mod\ q) \end{cases}$$

Применение аппарата эллиптических кривых. ECDSA.

Как было отмечено ранее, эллиптические кривые находят применение в криптографии. Причина их применения заключается в том, что данный математический аппарат позволяет получить более высокую эффективность безопасности на единицу времени выполнения.

Применение эллиптических кривых должно сопровождаться формированием параметров размещения (англ. domain parameters). Эти параметры включают значение q – основание конечного поля, параметры $a, b \in F_q$, задающие саму кривую, базовая точка $G \in E(F_q)$, число n, являющееся порядком базовой точки, фактор $h = \#E(F_q)/n$, где $\#E(F_q)$ – порядок кривой, составляющий число точек, принадлежащих ей.

Алгоритм генерации параметров области

Входные данные: основание поля q, L *– уровень безопасности. Рекомендуется выбирать* L: $160 \le L \le log_2 q$, $2^L \ge 4\sqrt{q}$

Выходные данные: параметры области $D = (q, a, b, G, n, h)$

1. Выбрать случайные $a, b \in F_q$, для которых $4a^3 + 27b^2 \not\equiv 0 \ (mod\ q)$
2. Вычислить порядок кривой $N = \#E(F_q)$
3. Убедиться, что $\exists n \ge 2^L$ такой, что $N\ mod\ n = 0$, иначе перейти к [1]
4. Убедиться, что $\forall k \in [1, 20]: q^k - 1 \equiv 0 \ (mod\ n)$, иначе перейти к [1]
5. Убедиться, что $n \ne q$, иначе перейти к [1]
6. Вычислить фактор $h = N/n$
7. Выбрать произвольную точку $G' \in E(F_q)$, вычислить $G = hG'$
8. Если $G = O$, повторить 7, иначе вернуть (q, a, b, G, n, h)

В рамках данного алгоритма проводится составная операция вычисления порядка кривой, в связи с чем, уместно использовать существующие параметры. Пример кривой приведен в Приложении 7.

Алгоритм генерации ключевой пары

Входные данные: параметры размещения D

Выходные данные: открытый ключ Q, закрытый ключ d

1. Выбрать случайное целое число $d \in [1, n-1]$
2. Вычислить точку $Q = dG$.
3. Вернуть (Q, d)

Здесь умножение nP определяется последовательным n-кратным сложением точки: $P + P + .. + P$. Эта операция может быть упрощена алгоритмом, аналогичным быстрому возведению в степень.

Алгоритм генерации подписи:

Входные данные: параметры размещения D, закрытый ключ d, сообщение m

Выходные данные: подпись (r,s)

1. Выбрать случайное целое $k \in [1, n-1]$
2. Вычислить $X = kG = (x_1, y_1)$
3. Вычислить $r = x_1 \ mod \ n$. Если $r = 0$, перейти к [1].
4. Вычислить $S = \mathrm{Prfsm}(m)$
5. Вычислить $s = k^{-1}(S + dr) \ mod \ n$. Если $s = 0$, перейти к [1]
6. Вернуть (r, s)

Здесь Prfsm – функция получения контрольной суммы.

Алгоритм проверки подписи:

Входные данные: параметры размещения D, открытый ключ Q, сообщение m, подпись (r, s)

Выходные данные: флаг достоверности подписи.

1. Проверить валидность r, s.

Если $r \notin [1, n-1]$ или $s \notin [1, n-1]$, вернуть "Отклонить"

2. Вычислить $S = \mathrm{Prfsm}(m)$
3. Вычислить $w = s^{-1} \ mod \ n$
4. Вычислить $u_1 = ew \ mod \ n$ и $u_2 = rw \ mod \ n$
5. Вычислить $X = (x_2, y_2) = u_1 G + u_2 Q$
6. Если $X = O$, вернуть "Отклонить", иначе вычислить $v = x_2 \ mod \ n$
7. Если $v = r$, вернуть "Принять", иначе "Отклонить"

Российские стандарты эллиптической криптографии.
ГОСТ 34.10-2018.

На территории Российской Федерации в 2018-м году был введён ряд стандартов в сфере криптографии, одним из которых выступил ГОСТ 34.10-2018, определяющий порядок формирования электронной цифровой подписи.

Параметрами алгоритма выступают следующие значения:
1. Простое число p – модуль эллиптической кривой, соответствующий неравенству $p > 2^{255}$
2. Эллиптическая кривая, заданная коэффициентами $a, b \in F_p$ или инвариантом $J(E) = 1728\frac{4a^3}{4a^3+27b^2}(mod\ p), 4a^3 + 27b^2 \neq 0\ (mod\ p)$
3. Целое число m – порядок группы точек эллиптической кривой, не равное p.
4. Простое число q, порядок циклической подгруппы точек эллиптической кривой. m = nq, для произвольного $n \in N$
$2^{254} < q < 2^{256}$
5. Точка P кривой E, являющаяся генератором подгруппы порядка q, $q * P = O, k * P \neq O, k = 1,2,..,q-1$
6. Функция контрольной суммы h(M) (как правило, ГОСТ 34.11-2018), отображающая сообщение M в двоичный вектор длины 256 бит.

Ключи в криптосистеме определяются следующим образом:
Скрытый ключ d: $0 < d < q$
Открытый ключ $Q = (x_Q, y_Q) = d * P$

Алгоритм формирования подписи:
1. z = h(M)
2. e = z mod q, при e = 0, задать e = 1
3. Генерация случайного k: $0 < k < q$
4. Вычисление C = k*P, r = x_C mod q.
5. При r = 0, вернуться к шагу 3.
6. Определение $s = (rd + ke)\ mod\ q$, при s = 0, вернуться к шагу 3.
Вернуть r, s как подпись

Алгоритм проверки подписи
1. . z = h(M)
2. e = z mod q, при e = 0, задать e = 1
3. $v = e^{-1} \bmod q$
4. $z_1 = sv \bmod q$; $z_2 = -rv \bmod q$
5. C = $z_1 P + z_2 Q; R = x_C \ mod \ q$
6. Проверить равенство R = r.

Если равенство соблюдается, вернуть “подпись верна”
Иначе “подпись некорректна”

Современные подходы к эллиптической криптографии. Кривые Эдвардса. EdDSA. Ed25519.

В 2007-м году профессором Гарольдом Эдвардсом было изучено семейство эллиптических кривых, давших толчок новым направлениям в эллиптической криптографии. Развитие идей Эдвардса получило успех в семействе алгоритмов, обозначаемых как EdDSA (Edwards Digital Signature Algorithm). В частности, самым распространённым вариантом выделяют Ed25519, построенную на основании $q = 2^{255} - 19$

Кривая Эдвардса выразима в следующем виде: $x^2 + y^2 = c^2(1 + x^2y^2)$

Даниэль Бернстайн и Тани Ланге развили идею этого класса кривых, выведя следующий родственный класс:
$x^2 + y^2 = c^2(1 + dx^2y^2), d - d^2c^4 \neq 0,$
$\left(\frac{d}{p}\right) = -1$ (d не является квадратичным вычетом p)
Алгоритм EdDSA определяется следующими параметрами:

- Конечное поле F_q и порядок q.
- Эллиптическая кривая E над полем F_q. Группа над кривой должна иметь порядок $\#E(F_q) = 2^c l$, где l – простое число с сильными алгебраическими свойствами, 2^c – кофактор кривой.
- Базовая точка B ∈ $E(F_q)$, имеющая порядок l.
- Криптографическая функция контрольной суммы H с 2b-битным выходом, где $2^{b-1} > q$.

Пара ключей генерируется следующим образом:
В роли закрытого ключа выступает b-битовая строка k, которая должна быть взята из равномерно генерируемых значений. Публичным ключом выступит точка A = sB, где s – первые b бит значения H(k), проинтерпретированные как целое число в LE (от младшего разряда к старшему).

Подпись (R,S) сообщения M формируется следующим образом:
R = rB, где r = H(PREFIX || M), где PREFIX = последние b бит H(k), || -- операция конкатенации. S = [r + H(R || A || M)s] mod l

Проверка подписи осуществляется следующим выражением:
2^cSB = 2^cR + 2^c H(R||A||M) A. Доказать равенство допустим проведя преобразования по вышеприведённым формулам [sB = A, rB = R, S = (r + H(R||A||M)s)]

При этом стоит учесть, что точки кривых Эдвардса слагаются иным образом, чем точки кривых, рассмотренных ранее. Для них формула выглядит следующим образом:

$$(x_1, y_1) + (x_2, y_2) = \left(\frac{x_1y_2 + x_2y_1}{1 + dx_1x_2y_1y_2}, \frac{y_1y_2 - x_1x_2}{1 - dx_1x_2y_1y_2}\right)$$

Где d – коэффициент при x^2y^2 в расширенной форме кривых Эдвардса.

Ed25519.

Достаточно распространённым экземпляром кривых Эдвардса выступает кривая на q = 2^{255} – 19.
Сама кривая задаётся соотношением $-x^2 + y^2 = 1 - \frac{121665}{121666}x^2y^2$.
l = 2^{252} + 27742317777372353535851937790883648493, c = 3
b = 256, H – SHA-512

Содержание работы

1. Разработать структурную единицу, предоставляющую возможность:
   1.1. Принимать параметры размещения.
   1.2. Генерировать ключевую пару на основании параметров.
   1.3. С использованием закрытого ключа – генерировать подпись.
   1.4. С использованием публичного ключа – проверять подпись.
2. Разработать приложение с графическим пользовательским интерфейсом, позволяющим применять функционал данной структурной единицы. В интерфейс должна входить возможность:
   2.1. Задавать параметры домена из файла.
   2.2. Сохранять в двух файлах генерируемые ключевые пары
   2.3. Сохранять в файл генерируемые цифровые подписи.
   2.4. Выводить на экран результат проверки цифровой подписи.
3. Осуществить тестирование приложения на основании тестовых данных, приведённых в Приложении 8.

Контрольные вопросы

1. Общее определение эллиптической кривой.
2. Понятие эллиптической группы.
   2.1. Операция сложения в группе
   2.2. Определение умножения в группе
   2.3. Нейтральный и обратные элементы
3. Конечная эллиптическая группа
   3.1. Аналитическое представление операции сложения
   3.2. Операция быстрого умножения
   3.3. Понятие порядка эллиптической кривой.
4. Принципы эллиптической криптографии.
   4.1. Проблема логарифма на кривой.
   4.2. Порядок генерации ключевой пары.
   4.3. Принцип формирования подписи.
   4.4. Принцип проверки подписи.

Приложение 1. Частоты символов русского алфавита.

а	0.0872	к	0.0373	х	0.0077
б	0.0190	л	0.0407	ц	0.0027
в	0.0446	м	0.0356	ч	0.0209
г	0.0190	н	0.0620	ш	0.0093
д	0.0333	о	0.1112	щ	0.0022
е	0.0882	п	0.0248	ъ	0.0002
ё	0.0001	р	0.0374	ы	0.0171
ж	0.0106	с	0.0502	ь	0.0222
з	0.0136	т	0.0695	э	0.0028
и	0.0592	у	0.0300	ю	0.0067
й	0.0097	ф	0.0012	я	0.0237

Таблица 1. Таблица частот

Примечание: Таблица составлена на основании текста книги Фёдора Михайловича Достоевского «Бедные люди».

Приложение 2. Таблицы SBox, InvSBox и массив RCon

```
Sbox = array{
0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76,
0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0,
0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x15,
0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75,
0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84,
0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf,
0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8,
0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2,
0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73,
0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb,
0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79,
0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08,
0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a,
0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e,
0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf,
0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16
}
```

Листинг 1. Массив Sbox

```
InvSbox = array{
0x52,0x09,0x6a,0xd5,0x30,0x36,0xa5,0x38,0xbf,0x40,0xa3,0x9e,0x81,0xf3,0xd7,0xfb,
0x7c,0xe3,0x39,0x82,0x9b,0x2f,0xff,0x87,0x34,0x8e,0x43,0x44,0xc4,0xde,0xe9,0xcb,
0x54,0x7b,0x94,0x32,0xa6,0xc2,0x23,0x3d,0xee,0x4c,0x95,0x0b,0x42,0xfa,0xc3,0x4e,
0x08,0x2e,0xa1,0x66,0x28,0xd9,0x24,0xb2,0x76,0x5b,0xa2,0x49,0x6d,0x8b,0xd1,0x25,
0x72,0xf8,0xf6,0x64,0x86,0x68,0x98,0x16,0xd4,0xa4,0x5c,0xcc,0x5d,0x65,0xb6,0x92,
0x6c,0x70,0x48,0x50,0xfd,0xed,0xb9,0xda,0x5e,0x15,0x46,0x57,0xa7,0x8d,0x9d,0x84,
0x90,0xd8,0xab,0x00,0x8c,0xbc,0xd3,0x0a,0xf7,0xe4,0x58,0x05,0xb8,0xb3,0x45,0x06,
0xd0,0x2c,0x1e,0x8f,0xca,0x3f,0x0f,0x02,0xc1,0xaf,0xbd,0x03,0x01,0x13,0x8a,0x6b,
0x3a,0x91,0x11,0x41,0x4f,0x67,0xdc,0xea,0x97,0xf2,0xcf,0xce,0xf0,0xb4,0xe6,0x73,
0x96,0xac,0x74,0x22,0xe7,0xad,0x35,0x85,0xe2,0xf9,0x37,0xe8,0x1c,0x75,0xdf,0x6e,
0x47,0xf1,0x1a,0x71,0x1d,0x29,0xc5,0x89,0x6f,0xb7,0x62,0x0e,0xaa,0x18,0xbe,0x1b,
0xfc,0x56,0x3e,0x4b,0xc6,0xd2,0x79,0x20,0x9a,0xdb,0xc0,0xfe,0x78,0xcd,0x5a,0xf4,
0x1f,0xdd,0xa8,0x33,0x88,0x07,0xc7,0x31,0xb1,0x12,0x10,0x59,0x27,0x80,0xec,0x5f,
0x60,0x51,0x7f,0xa9,0x19,0xb5,0x4a,0x0d,0x2d,0xe5,0x7a,0x9f,0x93,0xc9,0x9c,0xef,
0xa0,0xe0,0x3b,0x4d,0xae,0x2a,0xf5,0xb0,0xc8,0xeb,0xbb,0x3c,0x83,0x53,0x99,0x61,
0x17,0x2b,0x04,0x7e,0xba,0x77,0xd6,0x26,0xe1,0x69,0x14,0x63,0x55,0x21,0x0c,0x7d
}
```

Листинг 2. Массив InvSbox

```
RCON = array{
        0x00000000, 0x01000000, 0x02000000,
        0x04000000, 0x08000000, 0x10000000,
        0x20000000, 0x40000000, 0x80000000,
        0x1b000000, 0x36000000
}
```

Листинг 3. Массив Rcon

Приложение 3. Таблица мультипликативно-обратных значений в GF(2^8) с порождающим полиномом $g(x) = x^8 + x^4 + x^3 + x + 1$.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	00	01	8d	f6	cb	52	7b	d1	e8	4f	29	c0	b0	e1	e5	c7
1	74	b4	aa	4b	99	2b	60	5f	58	3f	fd	cc	ff	40	ee	b2
2	3a	6e	5a	f1	55	4d	a8	c9	c1	0a	98	15	30	44	a2	c2
3	2c	45	92	6c	f3	39	66	42	f2	35	20	6f	77	bb	59	19
4	1d	fe	37	67	2d	31	f5	69	a7	64	ab	13	54	25	e9	09
5	ed	5c	05	ca	4c	24	87	bf	18	3e	22	f0	51	ec	61	17
6	16	5e	af	d3	49	a6	36	43	f4	47	91	df	33	93	21	3b
7	79	b7	97	85	10	b5	ba	3c	b6	70	d0	06	a1	fa	81	82
8	83	7e	7f	80	96	73	be	56	9b	9e	95	d9	f7	02	b9	a4
9	de	6a	32	6d	d8	8a	84	72	2a	14	9f	88	f9	dc	89	9a
a	fb	7c	2e	c3	8f	b8	65	48	26	c8	12	4a	ce	e7	d2	62
b	0c	e0	1f	ef	11	75	78	71	a5	8e	76	3d	bd	bc	86	57
c	0b	28	2f	a3	da	d4	e4	0f	a9	27	53	04	1b	fc	ac	e6
d	7a	07	ae	63	c5	db	e2	ea	94	8b	c4	d5	9d	f8	90	6b
e	b1	0d	d6	eb	c6	0e	cf	ad	08	4e	d7	e3	5d	50	1e	b3
f	5b	23	38	34	68	46	03	8c	dd	9c	7d	a0	cd	1a	41	1c

Таблица 2. Таблица обратных значений в GF(2^8)

Приложение 4. Тестовые данные для лабораторной работы №3.

Исходный текст: $0123456789abcdeffedcba9876543210_{16}$

Основной ключ: $304870710e1420212223303334383940_{16}$

Раунд	Начало раунда	Результат SubBytes	Результат ShiftRows	Результат MixColumns	Ключ раунда
0	01 89 fe 76 23 ab dc 54 45 cd ba 32 67 ef 98 10				30 0e 22 34 48 14 23 38 70 20 30 39 71 21 33 40
1	31 87 dc 42 6b bf ff 6c 35 ed 8a 0b 16 ce ab 50	c7 17 86 2c 7f 08 16 50 96 55 7e 2b 47 8b 62 53	c7 17 86 2c 08 16 50 7f 7e 2b 96 55 53 47 8b 62	a0 78 fa ee 06 01 0c 4f c6 9e 67 5f 82 8a 5a 9a	36 38 1a 2e 5a 4e 6d 55 79 59 69 50 69 48 7b 3b
2	96 40 e0 c0 5c 4f 61 1a bf c7 0e 0f eb c2 21 a1	90 09 e1 ba 4a 84 ef a2 08 c6 ab 76 e9 25 fd 32	90 09 e1 ba 84 ef a2 4a ab 76 08 c6 32 e9 25 fd	35 a7 09 8a 57 bf 83 82 0f 2a 3c 7b e0 4b d8 b8	c8 f0 ea c4 09 47 2a 7f 9b c2 ab fb 58 10 6b 50
3	fd 57 e3 4e 5e f8 a9 fd 94 e8 97 80 b8 5b b3 e8	54 5b 11 2f 58 41 d3 54 22 9b 88 cd 6c 39 6d 9b	54 5b 11 2f 41 d3 54 58 88 cd 22 9b 9b 6c 39 6d	78 79 c5 40 ce c6 e6 44 a8 bd 4a ed 18 2b 37 68	1e ee 04 c0 06 41 6b 14 c8 0a a1 5a 44 54 3f 6f
4	66 97 c1 80 c8 87 8d 50 60 b7 eb b7 5c 7f 08 07	33 88 78 cd e8 17 5d 53 d0 a9 e9 a9 4a d2 30 c5	33 88 78 cd 17 5d 53 e8 e9 a9 d0 a9 c5 4a d2 30	73 0f 07 3b f8 98 67 d6 b9 42 fd 3c 3a e3 b4 6d	ec 02 06 c6 b8 f9 92 86 60 6a cb 91 fe aa 95 fa
5	9f 0d 01 fd 40 61 f5 50 d9 28 36 ad c4 49 21 97	db d7 7c 54 09 ef e6 53 35 34 05 95 1c 3b fd 88	db d7 7c 54 ef e6 53 09 05 95 35 34 88 1c 3b fd	0a 0d 03 7a 99 b8 be e7 bd 24 08 29 97 29 94 20	b8 ba bc 7a 39 c0 52 d4 4d 27 ec 7d 4a e0 75 8f
6	b2 b7 bf 00 a0 78 ec 33 f0 03 e4 54 dd c9 e1 af	37 a9 08 63 e0 bc ce c3 8c 7b 69 20 c1 dd f8 79	37 a9 08 63 bc ce c3 e0 69 20 8c 7b 79 c1 dd f8	a1 e1 1f 7e 96 8f c7 cd d2 7f b4 66 7e 97 f6 d5	d0 6a d6 ac c6 06 54 80 3e 19 f5 88 90 70 05 8a
7	71 8b c9 d2 50 89 93 4d ec 66 41 ee ee e7 f3 5f	a3 3d dd b5 53 a7 dc e3 ce 33 83 28 28 94 0d cf	a3 3d dd b5 a7 dc e3 53 83 28 ce 33 cf 28 94 0d	e3 05 c5 ba a7 ce dd 4b 53 c9 1e 97 5f e3 62 be	5d 37 e1 4d 02 04 50 d0 40 59 ac 24 01 71 74 fe
8	be 32 24 f7 a5 ca 8d 9b 13 90 b2 b3 5e 92 16 40	ae 23 36 68 06 74 5d 14 7d 60 37 6d 58 4f 47 09	ae 23 36 68 74 5d 14 06 37 6d 7d 60 09 58 4f 47	e5 94 62 fd 16 76 d6 83 af 4c 09 67 b8 e5 ad 50	ad 9a 7b 36 34 30 60 b0 fb a2 0e 2a e2 93 e7 19
10	48 0e 19 cb 22 46 b6 33 54 ee 07 4d 5a 76 4a 49	52 ab d4 1f 93 5a 4e c3 20 28 c5 e3 be 38 d6 3b	52 ab d4 1f 5a 4e c3 93 c5 e3 20 28 3b be 38 d6	b4 c2 f5 6e 89 b7 11 8c d4 e1 1f bd 1f 2c f4 2d	51 cb b0 86 d1 e1 81 31 2f 8d 83 a9 e7 74 93 8a
11	e5 09 45 e8 58 56 90 bd fb 6c 9c 14 f8 58 67 a7	d9 01 6e 9b 6a b1 60 7a 0f 50 de fa 41 6a 85 5c	d9 01 6e 9b b1 60 7a 6a de fa 0f 50 5c 41 6a 85		a0 6b db 5d 02 e3 62 53 51 dc 5f f6 a3 d7 44 ce
Итог	79 6a b5 c6 b3 83 18 39 8f 26 50 a6 ff 96 2e 4b				

Таблица 3. Тестовые данные для алгоритма AES.

Приложение 5. Таблица констант алгоритма SHA-256

```
k = array{
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
    0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
    0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc,
    0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
    0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13,
    0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
    0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5,
    0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208,
    0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
}
```

Листинг 4. Массив констант алгоритма SHA-256

Приложение 6. Тестовые данные для лабораторной работы №7

Исходные данные	Контрольная сумма в шестнадцатеричном виде
"What appears inconceivable may have a quite simple solution."	2df97ff534f52296 f3055cf4d408c591 7ed048ee3b5d6d04 154f053ecb4ca335
"There will be, in the next generation or so, a pharmacological method of making people love their servitude, and producing dictatorship without tears, so to speak, producing a kind of painless concentration camp for entire societies, so that people will in fact have their liberties taken away from them, but will rather enjoy it, because they will be distracted from any desire to rebel by propaganda or brainwashing, or brainwashing enhanced by pharmacological methods. And this seems to be the final revolution."	ab5c9c8ed5de4a93 e891e200827e08a8 a6d5555d800dd96a 1a703a5a203ac447
"As soon as completed, it will be possible for a .. man in New York to dictate instructions, and have them instantly appear in type at his office in London or elsewhere. He will be able to call up, from his desk, and talk to any telephone subscriber on the globe, without any change whatever in the existing equipment. An inexpensive instrument, not bigger than a watch, will enable its bearer to hear anywhere, on sea or land, music or song, the speech of a political leader, the address of an eminent man of science, or the sermon of an eloquent clergyman, delivered in some other place, however distant. In the same manner any picture, character, drawing, or print can be transferred from one to another place. Millions of such instruments can be operated from but one plant of this kind. More important than all of this, however, will be the transmission of power, without wires."	e77c49486ed4016d aaf944f6c228820c dc7513c7c685dfe1 5ba59fa0abd98ae5
1024 нулевых байтов	5f70bf18a0860070 16e948b04aed3b82 103a36bea41755b6 cddfaf10ace3c6ef
1024 единичных байтов	5a648d8015900d89 664e00e125df1796 36301a2d8fa191c1 aa2bd9358ea53a69
256 8-ми битных значений от 1-го до 255-ти и нуля	9bc038d0a0fb391f 3b33618dcf08b655 3560ef0ae0f7ad55 7871598f27b7194b

Таблица 4. Тестовые данные для SHA-256

№	n	e	d	M	S
1	6446fced478a 7485e31b5e89 43ecbd028d77 706ce1ec295f	6e9f5e881f1d bd5bba3b36a5 d42365b8c00c 91bbafd224b	111ccc2253cc 02907a7d8a25 3d74ed451d9a 39a6d4bb3f53	98748117011e 304817407071 0e1420303334 383940176371	4986f81a696e 3a799cddf067 4c8d7b5af049 cce50c7192bf
2	623f9c9e6b5a a022b679c812 2dc3b54603cf 9a90b6801245	de7f9d1a0b3e 023dfba1d914 e99f97b30fd0 e3cfcdefe91	58a645cc1d6a 8ca0b633f169 769bb6bc80fa cd48cb19bab5	333438394017 637198748117 011e30481740 70710e142030	37e3eaf8074b ed0c5ad11efe 0a7d34f8e693 c3feed9b5336
3	a51fa4e1f62c 15525cc1b69b 6ac154a44912 08a364d1f5fb	51f59f9ffdcf b02551e6af19 657b6ce6e7b0 f56c9fc05441	44de9127b61c 2e795052c735 67aedf4b5c1b 057383e472c1	98748117011e 304833343839 401763711740 70710e142030	791761f36a26 c5cab3478915 f45c58c5f8af 64577bf18bb6
4	8f18c0f7b425 987e38bb0b23 810ecdb68600 2850149cf739	6573ac244bb7 e86b0d42225d 189f63ffe3f5 68a6b21441fd	798f58b6c0cb 31a065c41ee8 8ef916356ab3 ff23d5147875	174070710e14 203033343839 401763719874 8117011e3048	36dcc19712c0 cfdd93eeb19d 5b36e87d3f2f b6e49c16e354
5	7b7dc2592e39 0ff79518ea96 3446d9e07eea 1a86bbe1b0f3	797f1b5e5d51 9aadf6cda581 92c294d248eb a5f8e0492441	2e1910a37246 43dfce90c482 d3058d3df47f b972b33162a1	333438394017 637117407071 0e1420309874 8117011e3048	6149a764327a 156b1d867359 ef8bf0f0507c c31547946830
6	cfc071923fb8 5f014e17b729 b15f90039a73 7f23d206959d	1c2f91e55eed 433eb44e19bb f0e397ac4fc4 ae06f2f6fe87	1582ddb38f95 028e6ae436b0 85e4fe061c69 d3eed17ba127	174070710e14 203098748117 011e30483334 383940176371	6c0805e9fe24 dceca5629c79 598a525be400 d9496e29964c

Таблица 5. Тестовые данные для ЭЦП алгоритма RSA

№	n	g	x	y	M	r	s
1	a146a6d69881 93a3b52cc23d badeea1f9d81 dde7e2fb53e7	1e9fcccd6a62 2ca1af349044 6f31fb477dcf 8f051cda5bd9	908c92fcd394 62c5ff9c939d 6971827cbcb5 d7dad5e63af5	9abf2913933e 8be6983e4881 65d01ab74fcf fa33b8cfb00d	98748117011e 304817407071 0e1420303334 383940176371	4c255def8244 23ee8a38b297 e674f151df1d 87545b9cddea	1804a2c01b6d 1ef27e9e2f2d 0ba400d54db7 4a6a60e36b46
2	ec05278621f1 bd39364d2123 1ea56ea5ef73 30bdf1ce2347	1df23b8b82c8 9b212e136954 00ce28915c78 cfdc172ef744	c40fe9e1ecb8 bed3f9c195b6 46aa81dac575 2b6d645e549d	d6db4e2c6df3 2929a716a685 c8ad690bdf8f a1c6e3c1534c	333438394017 637198748117 011e30481740 70710e142030	e3a163cf3494 4e70f2a40d8a 39a77c45215e c583cc2e4106	735995fe3174 d3dbe1e840db 65c07cfe4c70 b180e6a65062
3	baadbf81dd77 2a63cfa8f4aa 538fd6c40816 bdcb10830c5b	7a3553a384ae 5fc0054d0571 899aac4105e9 c27f634f58ef	8bfcecf285e3 c2ae109fb4f0 4df6ea718b7d fd8d6affc111	b882c7fc6ef7 68858420ad98 cff725848f1d 99e6683b5c23	98748117011e 304833343839 401763711740 70710e142030	1e16d100824f 24be83376976 8b1cf68b37c5 e1bb6a0238b5	8ceb4add7cad 2996c2f986de 13f39c5011b3 1a0164d8c32f
4	dc1bed3fd23a beb3bc8f9d61 e12f83bb83a8 70ad5deadf5b	11979e4c8d14 1461e1118cfd 3d5c8c2d603e e462530fe583	73c8648261dd 73d65fc59e5f e00b64179719 8475c73fc7b6	740fe35044b5 87c5bde899dd 82ee055b5dff 1e985f3bda0a	174070710e14 203033343839 401763719874 8117011e3048	8e946e653cf4 c898af625cb2 836fd83d8f8b afb97978d3b3	56daf6e59116 e44b2e36450b 254c91bc16c8 7a9465e7b298
5	c2b2a030131c ae3330bd23cd 3ff5015b1db3 faa304e9412f	8b570a60a575 45f73d899c83 30d1e99ceac3 b97aa7e6bede	21d70238f6d0 cc7eccd5e52a f5eba7c8f52a 9f0092c97eb0	5d7c32511a02 366d2328a267 8923c0cbd0cf 6c27fc3765aa	333438394017 637117407071 0e1420309874 8117011e3048	4774e300d87a bee8635c66b4 7a87bb104e6a 939c8859dd7f	aac4982d841d ed625fe2df3f 7074cdf81f5c b75c72aa6e26
6	de139d5307d4 1de480fc6149 38e01aea03e3 7ca3b410ae77	881a9cc816bc a045d34b5b2a d3f51aa8e557 e9fbc71af63f	374b631e6f1f ad5b96460746 5e4fa90c2a64 56ed5d3cddb8	1a5f38e03f43 4b6d88062032 d7a5abd828a5 64f28f7d7dd	174070710e14 203098748117 011e30483334 383940176371	a76d9a1ca516 6be84741fda0 0c63cb034ec8 e05dc8e9e0c8	a9429e081f6d 5a33dc3e4a01 8c0325adcda1 04cbad807289

Таблица 6. Тестовые данные для ЭЦП схемы Эль-Гамаля

Приложение 7. Параметры области для алгоритма ECDSA.

Кривая	q	a	b
P-256	ffffffff00000001 0000000000000000 00000000ffffffff ffffffffffffffff	ffffffff00000001 0000000000000000 00000000ffffffff fffffffffffffffc	5ac635d8aa3a93e7 b3ebbd55769886bc 651d06b0cc53b0f6 3bce3c3e27d2604b
Кривая	G_x	G_y	n
P-256	6b17d1f2e12c4247 f8bce6e563a440f2 77037d812deb33a0 f4a13945d898c296	ffffffff00000000 ffffffffffffffff bce6faada7179e84 f3b9cac2fc632551	4fe342e2fe1a7f9b 8ee7eb4a7c0f9e16 2bce33576b315ece cbb6406837bf51f5
Кривая	h	--	--
P-256	0000000000000000 0000000000000000 0000000000000000 0000000000000001	--	--

Таблица 7. Параметры области для ECDSA.

Приложение 8. Тестовые данные для алгоритма ECDSA.

Кривая	d	Qx	Qy
P-256	C477F9F65C22CCE2 0657FAA5B2D1D812 2336F851A508A1ED 04E479C34985BF96	B7E08AFDFE94BAD3 F1DC8C734798BA1C 62B3A0AD1E9EA2A3 8201CD0889BC7A19	3603F747959DBF7A 4BB226E419287290 63ADC7AE43529E61 B563BBC606CC5E09
Кривая	k	S	r
P-256	7A1A7E52797FC8CA AA435D2A4DACE391 58504BF204FBE19F 14DBB427FAEE50AE	A41A41A12A799548 211C410C65D8133A FDE34D28BDD542E4 B680CF2899C8A8C4	2B42F576D07F4165 FF65D1F3B1500F81 E44C316F1F0B3EF5 7325B69ACA46104F
Кривая	s		
P-256	DC42C2122D6392CD 3E3A993A89502A81 98C1886FE69D262C 4B329BDB6B63FAF1		

Таблица 8. Тестовые данные для ECDSA.